

Лабораторная работа № 7

«Создание веб-приложения для автоматизации расчета сопла Лавалья в OpenFOAM»

Аннотация

Данная работа направлена на создание веб-приложения для автоматического запуска расчета сопла Лавалья в OpenFOAM.

В ходе данного урока будет создано приложение, состоящее из двух форм: главная форма с исходными данными и форма с результатами расчета. Будут созданы bash скрипты для изменения параметров, рассмотрены разные способы запуска внешних функций из Node.js.

План занятия

- автоматизация изменения параметров OpenFOAM;
- создание главной формы приложения;
- создание формы с результатами;
- создание сервера;
- создание модальных окон.

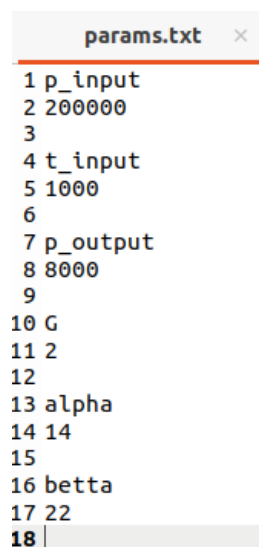
Форма представления выполненных заданий

Полученные в рамках данной лабораторной работы результаты прикладываются в качестве ответа в том же разделе LMS.

1 АВТОМАТИЧЕСКОЕ ИЗМЕНЕНИЕ ВСЕХ ФАЙЛОВ РЕШЕНИЯ OPENFOAM

В корневой директории `pozzle_1` необходимо создать текстовый файл, который будет хранить значения переменных, введенных пользователем. Назовите данный файл `params.txt`. В определенной последовательности запишем все входные параметры. После построчно будем считывать значения и изменять файлы OpenFOAM. В качестве начальных значений примем значения, представленные в листинге 1.

Листинг 1 – Файл с входными параметрами



```
params.txt x
1 p_input
2 200000
3
4 t_input
5 1000
6
7 p_output
8 8000
9
10 G
11 2
12
13 alpha
14 14
15
16 betta
17 22
18 |
```

Теперь создадим файл `ChangeParams.sh`. Можете скопировать уже имеющийся файл `*.sh` и его изменить, либо создать новый файл в терминале с помощью команды `touch ChangeParams.sh`. В последнем случае необходимо дать этому файлу право на выполнение: `chmod +x ChangeParams.sh`.

Для получения строк будем использовать следующие команды: `head`, которая показывает первые 10 строк, `tail` – последние 10 строк. В соответствующие переменные будем записывать значение на конкретной строке из файла `params.txt`. Командой `echo` выведем в терминал полученные значения.

Листинг 2 – Скрипт, считывающий входные данные

```
1 #!/bin/bash
2
3
4 p_input=$(head -n 2 params.txt | tail -n 1)
5 t_input=$(head -n 5 params.txt | tail -n 1)
6 p_output=$(head -n 8 params.txt | tail -n 1)
7 G=$(head -n 11 params.txt | tail -n 1)
8 alpha=$(head -n 14 params.txt | tail -n 1)
9 betta=$(head -n 17 params.txt | tail -n 1)
10
11 echo p_input = $p_input
12 echo t_input = $t_input
13 echo p_output = $p_output
14 echo G = $G
15 echo alpha = $alpha
16 echo betta = $betta
```

Введите в терминале команду для запуска скрипта: `./ChangeParams.sh`.

Вывод должен быть, как показано на рисунке 1.



```
sveta@sveta-VirtualBox:~/OpenFOAM/sveta-9/run/nozzle_1
$ ./ChangeParams.sh
p_input = 200000
t_input = 1000
p_output = 8000
G = 2
alpha = 14
betta = 22
```

Рисунок 1 – Результат работы ChangeParams.sh

Попробуйте изменить значения в файле `params.txt` и заново запустить скрипт.

Теперь напишем команду, которая будет изменять файл `p.org`. Воспользуемся командой `sed`. Откройте файл `p.org` и найдите строки, которые должны изменяться. В данном случае 18, 25 и 35 строки.

Листинг 3 – Начальные значения файла p.org

```
18 internalField uniform 9000;
19
20 boundaryField
21 {
22     inlet
23     {
24         type            fixedValue;
25         value            uniform 2e5;
26     }
27
28     outletRight
29     {
30         type            zeroGradient;
31     }
32     outletLeft
33     {
34         type            fixedValue;
35         value            uniform 9000;
36     }
```

В файле ChangeParams.sh добавляем следующую команду: `sed -i "18s/.*/internalField uniform ${p_output};;25s/.*/ value uniform ${p_input};;35s/.*/ value uniform ${p_output};" 0/p.org;`

Если у Вас нумерация иная, то вводите свои числа. Символы «.*» указывают на то, что будет изменяться вся строка, далее записывается новая строка, используем вычисленное значение. В конце путь к файлу. Попробуйте изменить значения давления на входе и выходе в файле params.txt и запустить скрипт.

По аналогии поступаем с файлом температуры. Добавляем следующую команду: `sed -i "18s/.*/internalField uniform ${t_input};;25s/.*/ value uniform ${t_input};;34s/.*/ value uniform ${t_input};" 0/T;` Проверьте свою нумерацию строк.

Теперь будем изменять файл начального распределения давления setFieldsDict, расположенный в каталоге system. Для начала напишем команду для изменения давления. Команду для изменения размеров области напишем позже. Добавьте в скрипт ChangeParams.sh еще одну команду: `sed -i "19s/.*/volScalarFieldValue p ${p_output};;29s/.*/ volScalarFieldValue p ${p_input};" system/setFieldsDict;` Проверьте нумерацию строк.

Следующей командой изменим входные параметры для построения геометрии и сетки в Salome. Откройте файл Mesh.py, расположенный в каталоге geometry. Найдите строки, в которых указаны входные параметры, их и будем изменять.

Листинг 4 – Входные параметры файла Mesh.py

```
13 # входные параметры
14 P_input = 200000
15 T_input = 1800
16 P_output = 9000
17 G = 1.5
18 alpha_ = 14
19 betta_ = 28
```

Напишите следующую команду: `sed -i "14s/.*/P_input = ${p_input};;15s/.*/T_input = ${t_input};;16s/.*/P_output = ${p_output};;17s/.*/G`

= $\{G\}/18s/.*/alpha_ = \{\alpha\}/19s/.*/beta_ = \{\beta\}/$ geometry/Mesh.py;.

Проверьте нумерацию строк и протестируйте.

Сейчас будем изменять размеры области с начальным распределением давления. Для этого в файле Mesh.py в самом конце добавим строки, которые будут открывать файл params.txt и записывать туда расчетную длину до критического сечения и высоту входного сечения. Данные параметры необходимо перевести в метры, а также входной диаметр разделить на 2, так как нас интересует радиус.

Листинг 5 – Запись данных для начального распределения

```
570 # change params for setFields
571 file = open("/home/sveta/OpenFOAM/sveta-9/run/nozzle_1/params.txt", "a")
572 file.write(f"width\n{l_left/1000}\n\nheight\n{d_inlet_/2000}")
573 file.close()
```

В терминале выполните команду по построению сетки, затем проверьте файл params.txt, в нем должны появиться следующие строки:

Листинг 6 – Добавленные строки в файл params.txt

```
18
19 width
20 1.1850024052703034
21
22 height
23 0.1955
```

Теперь можно считывать их и изменять область начального распределения давления. Перед командой setFields в файле Run.sh добавляем команду для изменения размеров области.

Листинг 7 – Изменение области начального распределения

```
11 width=$(head -n 20 params.txt | tail -n 1)
12 height=$(head -n 23 params.txt | tail -n 1)
13
14
15 sed -i "26s/.*/          box (-${width} 0 -1) (0 ${height} 1);/" system/-
    setFieldsDict;
16
17 setFields
```

Также важно очищать полученные строки в файле params.txt, поэтому в файл Clean.sh добавляем команду: sed -i '19,23d' params.txt, которая удалит с 19 по 23 строки в файле params.txt.

Добавим вывод расчета в файл log, для этого достаточно написать: >log после запуска решателя в файле Run.sh.

Листинг 8 – Добавление вывода решения в файл log

```
19 mpirun -np 4 rhoPimpleFoam -parallel >log
```

Добавляем команду удаления файла log в Clean.sh: rm log.

Теперь последовательно вызываем скрипты Clean.sh, ChangeParams.sh, Run.sh. Ход решения будет записываться в файл log. Проверьте, что все выполняется без ошибок.

Добавьте в файлы Clean.sh и ChangeParams.sh переход в текущую директорию, так как вызываться эти скрипты должны именно из нее.

Листинг 9 – Переход в директорию проекта

```
1 #!/bin/bash
2
3 cd /home/sveta/OpenFOAM/sveta-9/run/nozzle_1
```

Также можно не построчно изменять значения в соответствующих файлах с помощью команды sed, а подключить в каждый файл импорт params.txt, откуда и будут взяты соответствующие значения.

2 СОЗДАНИЕ ВЕБ ИНТЕРФЕЙСА

2.1 Создание базового приложения

В домашней директории создайте каталог `nozzle_1`, в котором будет создано приложение. Оно будет разделено на серверную и клиентскую части. Откройте `nozzle_1` в Visual Studio Code. Создайте папку `client` (Рисунок 2).

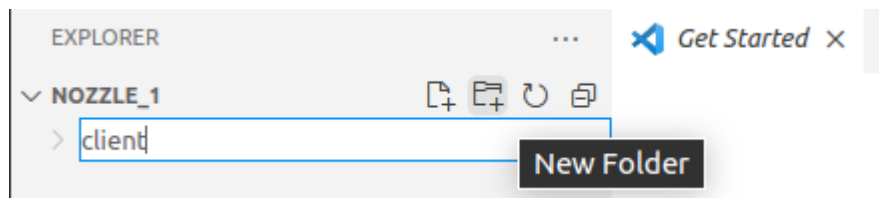


Рисунок 2 – Создание каталога `client`

Откройте новый терминал. Для начала создадим базовое приложение на Vue3, используя CLI. На официальном сайте vuejs.org в разделе документации можно найти команду для создания приложения. Vue/cli должно быть установлено у Вас в системе глобально, установите последнюю версию. Команда может отличаться, поэтому на официальном сайте найдите актуальную.

В терминале переходим в папку `client`: `cd client/`. Вводим команду для создания приложения: `vue create .`; точка говорит о том, что проект будет создан в данном каталоге. Далее будет следовать ряд вопросов. Ответьте утвердительно на создание приложения в текущей директории и выберите предложенный выбор по умолчанию: [Vue 3] `babel`, `eslint`. Ожидайте создания приложения. После откройте файл `package.json`. Сразу добавим зависимости, которые потребуются во время создания приложения. В терминале введите команду: `npm i axios cors vue-router`. `Axios` нужен для отправки запросов на сервер; `cors` – для того, чтобы оправа не блокировалась, так как браузер не разрешает по умолчанию отправлять запросы с другого источника; `vue-router` – для маршрутов.

Попробуйте запустить приложение, введите команду: `npm run serve`. На `http://localhost:8080/` должно запуститься базовое приложение Vue. Рекомендуется установить расширение `Vetur` для комфортной работы с Vue.

2.2 Изменение корневого файла App.vue

Теперь перейдем к созданию приложения. Оно будет состоять из двух страниц. Первая для ввода входных параметров, вторая для отображения хода решения и управления расчетом.

Начнем с корневого файла App.vue. В нем будут расположены заголовок и кнопка для начала нового расчета. В каталоге src найдите соответствующий файл и удалите все, что находится в теге `<template>`, удалите импорт компонента HelloWorld, регистрацию компонента. Создайте тег `<nav>`, который будет содержать заголовок `<h3>` и кнопку для запуска нового расчета. Кнопке задайте класс `btn`. Добавьте `<router-view />` для отображения соответствующих страниц.

Листинг 10 – Файл App.vue

```
<template>
  <nav>
    <h3>Расчет сопла Лавалья</h3>
    <button class="btn">
      Новый расчет
    </button>
  </nav>
  <router-view />
</template>

<script>

export default {
  name: 'App',
}
</script>
```

В теге `<style>` изменим стили для данной страницы. Вы можете использовать свои стили, так как они влияют только на внешний вид. У тега `<style>` не стоит атрибут `scoped`, следовательно, данные стили будут доступны и остальным компонентам.

Листинг 11 – Стили компонента App.vue

```
<style>
#app {
  font-family: Trebuchet MS, Lucida Grande, Lucida Sans Unicode, Lucida Sans,
  | Tahoma, sans-serif;
  text-align: center;
}

nav {
  padding: 10px;
  margin: 0;
  background-color: ■rgba(0, 0, 0, 0.89);
}

h3 {
  text-align: center;
  color: □white;
  font-size: 30px;
  padding: 5px;
  padding-right: 50px;
  display: inline-block;
}
```

Для кнопки также зададим определенные стили и ее отображение при наведении.

Листинг 12 – Стили компонента App.vue продолжение

```
button {
  color: □#fff;
  background: linear-gradient(to right, ■#11998e, ■#38ef7d);
  outline: none;
  border: 0;
  color: □#fff;
  padding: 10px 20px;
  text-transform: uppercase;
  border-radius: 2px;
  cursor: pointer;
  position: relative;
  width: 200px;
  display: inline-block;
}

.btn {
  margin-left: 16rem;
}

button:hover {
  box-shadow: 3px 4px 20px ■#828282;
}
</style>
```

Сохраните изменения и посмотрите на свое приложение в браузере. Введите команду `npm run serve`, если останавливали процесс в терминале, если нет, то страница должна обновиться автоматически, благодаря `nodemon`.

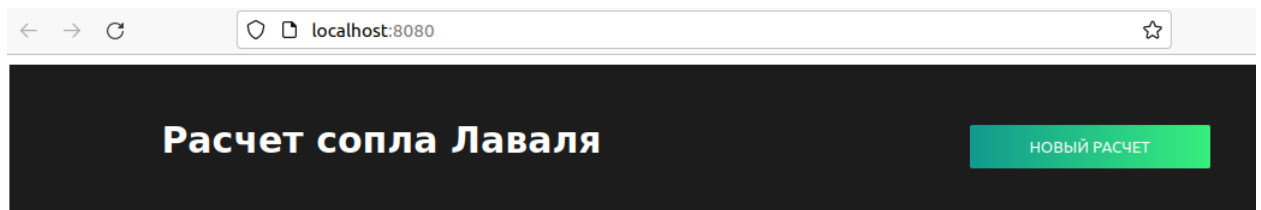


Рисунок 3 – Компонент App.vue в браузере

2.3 Создание главной формы приложения

В папке src создадим каталог views для создания в нем двух страниц: главная и результаты.

Первую назовем HomeView.vue. В блоке <div> будет расположен один компонент HomePage. Его необходимо импортировать из соответствующей директории и зарегистрировать в объекте компонентов.

Листинг 13 – Компонент HomeView.vue

```
<template>
  <div>
    <HomePage />
  </div>
</template>

<script>
import HomePage from "@/components/HomePage.vue";

export default {
  name: "HomeView",
  components: {
    HomePage,
  },
};
</script>
```

Теперь создадим данный компонент в папке components. Удалите компонент HelloWorld.vue, он не понадобится.

Главная форма будет состоять из двух блоков: слева расположим изображение сопла Лаваля с постоянными данными для расчета: среда, газовая постоянная воздуха и показатель адиабаты; справа будет форма ввода входных данных и кнопка запуска расчета.

В компоненте HomePage.vue создайте базовую разметку. Укажите корневой <div> с классом app. В секции script заполните только поле name. Всю разметку создавать на данной странице не будем, разделим на два компонента: карточка и форма ввода.

Листинг 14 – Базовая разметка компонента HomePage.vue

```
<template>
  <div id="app">
  </div>
</template>

<script>

export default {
  name: "HomePage",
  data(){
  }
}
</script>

<style scoped>

</style>
```

Теперь создайте новый компонент CardComponent.vue, который позже импортируем в компонент HomePage.vue. Он состоит из корневого блока <div> с классом container, содержащий в себе карточку с исходными данными для расчета и изображение сопла Лавая. Добавьте заголовок <h3> и тег <section>, содержащий в теге параграфа <p> информацию о постоянных параметрах для расчета. Для переноса на новую строку используйте тег
. Тегу укажите атрибут id для стилизации компонента. Путь до изображения записывается в атрибуте src.

В каталог assets добавьте изображение сопла Лавая, проверьте название – Soplo.png. В секции script задайте имя компоненту – CardComponent.

Листинг 15 – Файл CardComponent.vue

```
<template>
  <div class="container">
    <div class="card">
      <h3 class="card--title">Исходные данные для расчета</h3>
      <section class="card__content">
        <p>
          1. Среда - воздух. <br />
          2. Газовая постоянная воздуха  $R = 287$  Дж/(кг*К).
          <br />
          3. Показатель адиабаты  $k = 1.4$ .
        </p>
      </section>
    </div>
    
  </div>
</template>

<script>

export default {
  name: "CardComponent",
  data() {
  },
};
</script>
```

Добавите стили для элементов.

Листинг 16 – Стили компонента CardComponent.vue

```
@import url("https://fonts.googleapis.com/css?family=Noto+Sans:400,400i,700,700i&subset=greek-ext");

.container {
  display: inline-block;
  width: 50%;
  overflow: hidden;
  background-size: cover;
  padding: 30px;
  vertical-align: top;
}
#img {
  width: 100%;
  height: 80%;
  margin-top: -7rem;
}
.card {
  border-radius: 3px;
  box-shadow: -10px -5px 5px #d3d3d3;
  padding: 20px 30px;
  padding-bottom: 10rem;
  width: 100%;
  margin: 0 0;
}

.card--title {
  text-align: center;
  font-weight: 90;
}
```

Листинг 17 – Стили компонента CardComponent.vue продолжение

```
.card_content {  
  padding: 10px;  
  box-sizing: border-box;  
  line-height: 2;  
}  
.card_content p {  
  color: black;  
}
```

Сохраните изменения. Теперь в компонент HomePage.vue необходимо импортировать созданный компонент карточки. Зарегистрируйте его и добавьте в <div> с id = app.

Листинг 18 – Импорт компонента CardComponent.vue

```
<template>  
  <div id="app">  
    <CardComponent />  
  </div>  
</template>  
  
<script>  
import CardComponent from "@components/CardComponent.vue"  
  
export default {  
  name: "HomePage",  
  components: {  
    CardComponent,  
  },  
  data() {
```

Теперь необходимо определить путь, по которому будет отображаться страница HomeView.vue. В каталоге src создайте папку router, содержащую файл index.js с определенными маршрутами. По пути «/» будет отображаться страница HomeView.vue. Создайте массив объектов, укажите путь, имя и соответствующий компонент. Импортируйте HomeView из каталога views. Экспортируйте созданный роутер по умолчанию.

Листинг 19 – Создание маршрута

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'

const routes = [
  {
    path: '/',
    name: 'home',
    component: HomeView
  },
]

const router = createRouter({
  history: createWebHistory(process.env.BASE_URL),
  routes
})

export default router
```

Измените файл main.js. Добавьте импорт и использование роутера.

Листинг 20 – Добавление использования роутера

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'

createApp(App).use(router).mount('#app')
```

Сохраните изменения во всех файлах и проверьте страницу в браузере (Рисунок 4).

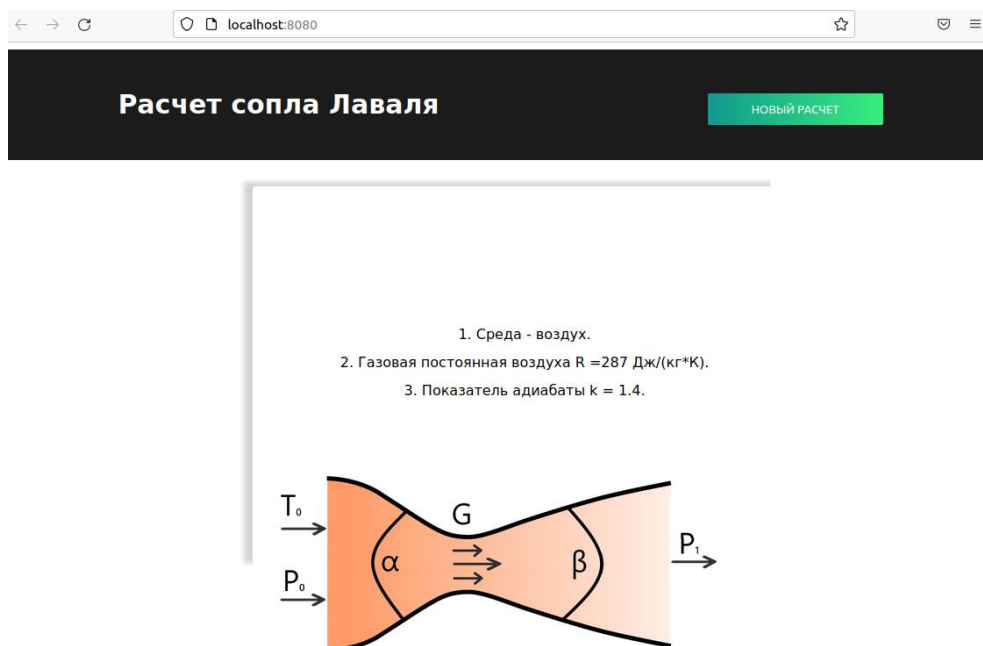


Рисунок 4 – Главная форма приложения

Теперь создадим форму с вводом входных данных. В каталоге components создайте новый компонент, назовите его FormComponent.vue. В секции <template> создайте блок <div> с классом box, который будет содержать форму ввода входных параметров. В атрибут method установите значение post, атрибут action оставьте пока пустым, позже в него запишем адрес, по которому будут передаваться данные. В секции script укажите поле name.

Листинг 21 – Базовая разметка компонента FormComponent.vue

```
<template>
  <div class="box">
    <form action="" id="app" method="post">

    </form>
  </div>
</template>

<script>

export default {
  name: "FormComponent",
  data() {

  },
};
</script>

<style scoped>

</style>
```

Входными параметрами являются следующие:

- давление и температура на входе,
- давление на выходе,
- расход воздуха,
- углы раскрытия сопла (входная и выходная части).

Создайте их и укажите начальные значения.

Листинг 22 – Создание моделей компонента FormComponent.vue

```
export default {  
  name: "FormComponent",  
  data() {  
    return {  
      P_input: 200000,  
      T_input: 1000,  
      P_output: 8000,  
      G: 2.0,  
      Alpha: 14,  
      Betta: 22,  
    };  
  },  
};  
</script>
```

Теперь создадим форму. Для начала добавьте заголовок в теге `<h2>`. Создайте блок `<div>` с классом `input-container`, который будет содержать поле для ввода и название в теге `<label>`. Тегу `<input>` необходимо добавить атрибуты: тип (число), имя (для отправки данных), модель (двустороннее связывание с моделью из `data`), минимальное значение, обязательность заполнения данного поля перед отправкой.

Листинг 23 – Создание поля ввода давления на входе

```
<form action="" id="app" method="post">  
  <h2>Входные параметры:</h2>  
  <div class="input-container">  
    <input  
      type="number"  
      name="p_input"  
      v-model="P_input"  
      min="1"  
      required  
    />  
    <label>Давление воздуха <b>P0</b> на входе в сопло (Па)</label>  
  </div>  
</form>
```

Максимальное значение не указываем, так как, например, в зависимости от разного давления на входе возможные значения расхода воздуха будут отличаться. Ошибки будут отслеживаться в ходе выполнения расчета.

Добавьте стили для созданных элементов.

Листинг 24 – Стили компонента FormComponent.vue

```
<style scoped>
h2 {
  color: ■rgb(0, 0, 0);
  padding-bottom: 40px;
}
.box {
  background-color: □white;
  padding: 0 30px;
  padding-bottom: 30px;
  display: inline-block;
  width: 45%;
  overflow: hidden;
  background-size: cover;
  box-shadow: 10px 5px 5px □#d3d3d3;
  margin-right: 10px;
  margin-top: 40px;
}
.input-container {
  position: relative;
  margin-bottom: 40px;
}
.input-container label {
  position: absolute;
  top: -14px;
  left: 0px;
  font-size: 14px;
  color: ■rgb(123, 123, 123);
  transition: all 0.5s ease-in-out;
}
.input-container input {
  border: 0;
  border-bottom: 1px solid ■#555;
  background: transparent;
  width: 100%;
  padding: 8px 0 5px 0;
  font-size: 16px;
  color: ■rgb(0, 0, 0);
}
.input-container input:focus {
  outline: none;
  border-width: 1px;
  border-image: linear-gradient(to right, ■#11998e, ■#38ef7d);
  border-image-slice: 1;
}
.input-container input:focus ~ label,
.input-container input:valid ~ label {
  color: ■rgb(0, 0, 0);
  top: -18px;
  font-size: 16px;
}
</style>
```

В начало добавьте стили для установки шрифтов и начальных отступов как в компонент FormComponent, так и в компонент CardComponent.

Листинг 25 – Добавление базовых стилей в FormComponent и CardComponent

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: Trebuchet MS, Lucida Grande, Lucida Sans Unicode, Lucida Sans,
    Tahoma, sans-serif;
}
```

Сохраните изменения.

В файл HomePage импортируйте созданный компонент, зарегистрируйте его и добавьте в блок <div>.

Листинг 26 – Импорт компонента в файл HomePage.vue

```
<template>
  <div id="app">
    <CardComponent />
    <FormComponent />
  </div>
</template>

<script>
import CardComponent from "@components/CardComponent.vue"
import FormComponent from "@components/FormComponent.vue"

export default {
  name: "HomePage",
  components: {
    CardComponent,
    FormComponent
  },
}
```

Сохраните изменения, проверьте работу приложения в браузере. У вас должно получиться, как на рисунке 5.

Теперь главная страница содержит как компонент CardComponent, так и FormComponent.

1. Среда - воздух.

2. Газовая постоянная воздуха $R = 287$
Дж/(кг*К).

3. Показатель адиабаты $k = 1.4$.

Входные параметры:

Давление воздуха P_0 на входе в сопло (Па)

Рисунок 5 – Добавление поля ввода давления на входе в компонент FormComponent.vue

По аналогии добавьте оставшиеся поля для ввода входных параметров.

Листинг 27 – Добавление блока с вводом температуры на входе в сопло.

```
<div class="input-container">
  <input
    type="number"
    class="inputbox"
    name="t_input"
    v-model="T_input"
    min="1"
    required
  />
  <label>Температура воздуха <b>T0</b> на входе в сопло (K)</label>
</div>
```

Листинг 28 – Добавление блока с вводом давления на выходе из сопла.

```
<div class="input-container">
  <input
    type="number"
    class="inputbox"
    name="p_output"
    v-model="P_output"
    min="1"
    required
  />
  <label>Давление воздуха <b>P1</b> на выходе из сопла (Па)</label>
</div>
```

Листинг 29 – Добавление блока с вводом расхода воздуха через сопло.

```
<div class="input-container">
  <input
    type="number"
    class="inputbox"
    name="g"
    v-model="G"
    min="0"
    required
  />
  <label>Расход воздуха <b>G</b> через сопло (Kg/c)</label>
</div>
```

Листинг 30 – Добавление блока с вводом угла раскрытия сопла – входная часть.

```
<div class="input-container">
  <input
    type="number"
    class="inputbox"
    name="alpha"
    v-model="Alpha"
    min="1"
    max="89"
    required
  />
  <label>Угол раскрытия сопла <b> $\alpha$ </b> - входная часть (°)</label>
</div>
```

Укажите также максимальное значение. Угол не может быть больше либо равным 90°.

Листинг 31 – Добавление блока с вводом угла раскрытия сопла – выходная часть.

```
<div class="input-container">
  <input
    type="number"
    class="inputbox"
    name="beta"
    v-model="Betta"
    min="1"
    max="89"
    required
  />
  <label>Угол раскрытия сопла <b> $\beta$ </b> - выходная часть (°)</label>
</div>
```

В конце создайте кнопку с типом submit и классом btn.

Листинг 32 – Создание кнопки «Расчет»

```
<button type="submit" class="btn">Расчет</button>
```

Добавьте стили для кнопки.

Листинг 33 – Стили для кнопки

```
.btn {  
  color: □#fff;  
  background: linear-gradient(to right, ■#11998e, ■#38ef7d);  
  outline: none;  
  border: 0;  
  padding: 10px 20px;  
  text-transform: uppercase;  
  border-radius: 2px;  
  cursor: pointer;  
  position: relative;  
}
```

Сохраните изменения и проверьте свое приложение в браузере. Оно должно выглядеть, как показано на рисунке 6.

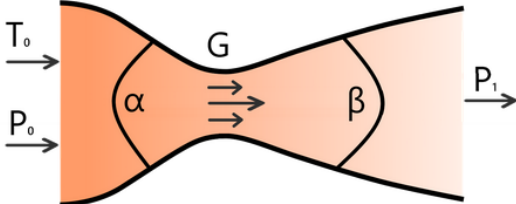
Расчет сопла Лавала

НОВЫЙ РАСЧЕТ

1. Среда - воздух.

2. Газовая постоянная воздуха $R = 287$
Дж/(кг*К).

3. Показатель адиабаты $k = 1.4$.



Входные параметры:

Давление воздуха P_0 на входе в сопло (Па)
200000

Температура воздуха T_0 на входе в сопло (К)
1000

Давление воздуха P_1 на выходе из сопла (Па)
8000

Расход воздуха G через сопло (Кг/с)
2

Угол раскрытия сопла α - входная часть (°)
14

Угол раскрытия сопла β - выходная часть (°)
22

РАСЧЕТ

Рисунок 6 – Главная форма приложения

Проверьте, что кнопка «Расчет» не работает, пока все поля не заполнены. Но на данный момент в поля можно вписывать не только цифры. Создайте событие нажатия на клавишу для каждого <input>. Назовите его NumbersOnly.

Листинг 34 – Добавление события keypress

```
<input
  type="number"
  name="p_input"
  v-model="P_input"
  @keypress="NumbersOnly"
  min="1"
  required
/>
```

На одном уровне со свойством data в секции script создайте опцию methods, содержащую метод NumbersOnly, который позволит вводить только цифры и точку. Он отслеживает коды нажатых клавиш.

Листинг 35 – Создание метода NumbersOnly

```
methods: {
  NumbersOnly(evt) {
    evt = (evt) ? evt : window.event;
    var charCode = (evt.which) ? evt.which : evt.keyCode;
    if ((charCode > 31 && (charCode < 48 || charCode > 57)) && charCode !== 46) {
      evt.preventDefault();
    } else {
      return true;
    }
  }
};
</script>
```

Проверьте работу метода.

2.4 Создание формы с результатами

Теперь создадим вторую форму, на которой будут отображаться данные, с которыми был запущен расчет, работа промежуточных функций и время, для которого расчет был выполнен. Также будут созданы три кнопки: «Остановить расчет», «Открыть ParaView для просмотра результатов», «Открыть файл log». Последний будет создаваться OpenFOAM во время расчета.

В каталоге views создайте новый файл ResultView.vue. По аналогии с файлом HomeView.vue импортируйте в него страницу с результатами, зарегистрируйте данный компонент и добавьте в блок <div> для отображения на странице.

Листинг 36 – Компонент ResultView

```
<template>
  <div>
    <ResultPage />
  </div>
</template>

<script>
import ResultPage from "@/components/ResultPage.vue";

export default {
  name: "ResultView",
  components: {
    ResultPage,
  },
};
</script>
```

Создайте компонент ResultPage.vue в каталоге components. Данная страница будет состоять из одной карточки, разделенной на две части. Левая будет выводить данные, с которыми был запущен расчет, и 3 кнопки. Правая будет содержать информацию о результатах выполнения функций до начала расчета и результаты, которые уже готовы для анализа. Создайте базовую разметку.

Листинг 37 – Базовая разметка компонента ResultPage.vue

```
<template>
  <div id="app">
    <div class="main">
      <div class="card">
        <div class="leftside">

        </div>
        <div class="rightside">

        </div>
      </div>
    </div>
  </div>
</template>
```

Добавьте определенные стили для созданных элементов.

Листинг 38 – Стили компонента ResultPage.vue

```
body {
  font-family: Trebuchet MS, Lucida Grande, Lucida Sans Unicode, Lucida Sans,
    Tahoma, sans-serif;
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

.main {
  min-height: 100vh;
  width: 100%;
  display: flex;
  flex-direction: column;
  background-color: #ffffff;
  color: #1f121b;
}

.card {
  width: 70rem;
  margin: auto;
  margin-top: 50px;
  background: white;
  position: center;
  align-self: center;
  top: 50rem;
  box-shadow: 4px 3px 20px #3535358c;
  display: flex;
  flex-direction: row;
}
```

Листинг 39 – Стили компонента ResultPage.vue продолжение

```
.leftside {
  background-color: #ffffff;
  width: 35rem;
  padding: 1rem 2rem 5rem 3rem;
  text-align: center;
}

.rightside {
  background-color: #ffffff;
  width: 40rem;
  padding: 1rem 2rem 5rem 3rem;
  box-shadow: 0px 0px 4px #3535358c;
  text-align: center;
}
```

Сохраните изменения. Теперь добавим путь, по которому данная страница будет отображаться.

Перейдите в файл index.js каталога router. Добавьте еще один маршрут, импортируйте ResultView.

Листинг 40 – Определение пути отображения страницы результатов

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'
import ResultView from '../views/ResultView.vue'

const routes = [
  {
    path: '/',
    name: 'home',
    component: HomeView
  },
  {
    path: '/result',
    name: 'result',
    component: ResultView
  }
]
```

Сохраните изменения и откройте страницу в браузере. По пути /result должна отображаться пустая карточка, разделенная на две части (Рисунок 7).

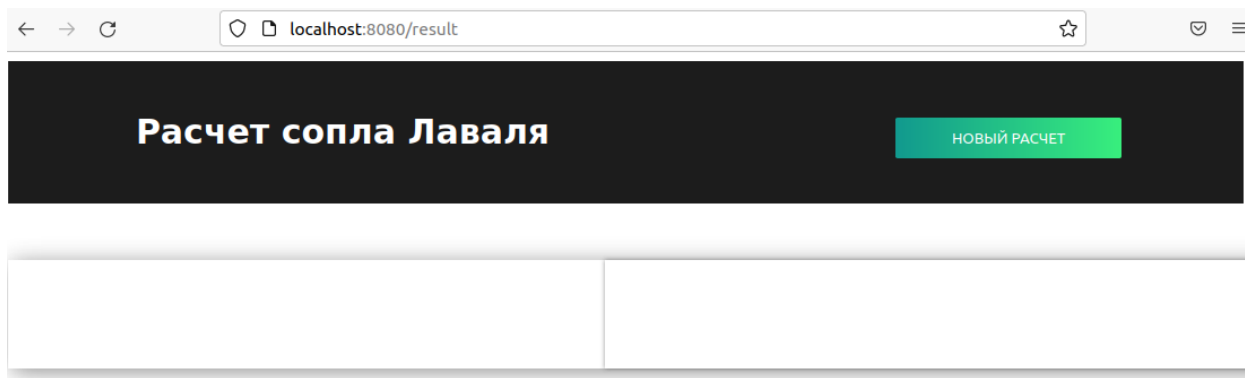


Рисунок 7 – Отображение страницы ResultView

Теперь заполним левую часть карточки. В самом начале будет отображаться информация о входных параметрах, с которыми был запущен расчет. С помощью интерполяции (`{{ }}`) будут выводиться введенные пользователем данные.

Листинг 41 – Информация о входных данных для расчета

```
<div class="leftside">
  <h2>Был запущен расчет со следующими входными параметрами:</h2>
  <p>
    Давление воздуха <b>P0</b> на входе в сопло:
    <b>{{ P_input }}</b> (Па)
  </p>
  <p>
    Температура воздуха <b>T0</b> на входе в сопло:
    <b>{{ T_input }}</b> (К)
  </p>
  <p>
    Давление воздуха <b>P1</b> на выходе из сопла:
    <b>{{ P_output }}</b> (Па)
  </p>
  <p>
    Расход воздуха <b>G</b> через сопло: <b>{{ G }}</b> (Кг/с)
  </p>
  <p>
    Угол раскрытия сопла <b>α</b> - входная часть:
    <b>{{ Alpha }}</b> (°)
  </p>
  <p>
    Угол раскрытия сопла <b>β</b> - выходная часть:
    <b>{{ Betta }}</b> (°)
  </p>
</div>
```

Далее создайте три кнопки, добавьте им класс button.

Листинг 42 – Создание кнопок управления расчетом

```
<button class="button">Остановить расчет</button>
<button class="button">
  Открыть ParaView для просмотра текущих результатов
</button>
<button class="button">
  Открыть файл log для просмотра решения
</button>
</div>
```

В свойстве data создайте модели, которые будут хранить в себе входные параметры.

Листинг 43 – Создание моделей

```
name: "ResultPage",
data() {
  return {
    P_input: "",
    T_input: "",
    P_output: "",
    G: "",
    Alpha: "",
    Beta: "",
  }
},
```

Добавьте стили для параграфа и кнопок.

Листинг 44 – Добавление стилей

```
p {
  margin: 0.1rem 0;
  font-size: 1.1rem;
  font-weight: 400;
}
.button {
  color: #fff;
  background: linear-gradient(to right, #11998e, #38ef7d);
  outline: none;
  border: 0;
  color: #fff;
  padding: 10px 20px;
  text-transform: uppercase;
  border-radius: 2px;
  cursor: pointer;
  position: relative;
  min-width: 500px;
  margin-top: 20px;
}
```

Сохраните изменения и проверьте страницу в браузере. Должно получиться, как на рисунке 8.

Расчет сопла Лавая НОВЫЙ РАСЧЕТ

Был запущен расчет со следующими входными параметрами:

Давление воздуха P_0 на входе в сопло: (Па)
Температура воздуха T_0 на входе в сопло: (К)
Давление воздуха P_1 на выходе из сопла: (Па)
Расход воздуха G через сопло: (Кг/с)
Угол раскрытия сопла α - входная часть: (°)
Угол раскрытия сопла β - выходная часть: (°)

ОСТАНОВИТЬ РАСЧЕТ

ОТКРЫТЬ PARAVIEW ДЛЯ ПРОСМОТРА ТЕКУЩИХ РЕЗУЛЬТАТОВ

ОТКРЫТЬ ФАЙЛ LOG ДЛЯ ПРОСМОТРА РЕШЕНИЯ

Рисунок 8 – Измененная левая часть формы с результатами

Данные, которые пользователь введет, будут записываться в соответствующие модели при загрузке страницы.

2.5 Создание сервера

Откройте новый терминал. В корне директории на одном уровне с каталогом client создайте каталог server. Перейдите в него в терминале: `cd server`. Проинициализируйте новый проект с дефолтными настройками: `npm init -y`. Должен появиться файл `package.json`. Теперь установим необходимые зависимости. В качестве зависимости только на этапе разработки установим `nodemon`, который будет автоматически перезагружать сервер: `npm i -D nodemon`. В разделе `devDependencies` должен появиться установленный пакет.

Листинг 45 – Установленный пакет `nodemon`

```
"keywords": [],  
"author": "",  
"license": "ISC",  
"devDependencies": {  
  "nodemon": "^2.0.15"  
}
```

Теперь установим пакеты `express`, `cors` и `body-parser`: `npm i express cors body-parser`. Данные пакеты должны появиться в разделе `dependencies`, так как они нужны не только на этапе разработки.

Листинг 46 – Установленные зависимости

```
"dependencies": {  
  "body-parser": "^1.20.0",  
  "cors": "^2.8.5",  
  "express": "^4.17.3"  
}
```

Создайте скрипты для запуска сервера. На этапе разработки используем `nodemon`, поэтому будем выполнять команду: `npm run dev`.

Листинг 47 – Скрипты для запуска сервера

```
"scripts": {  
  "start": "node index.js",  
  "dev": "nodemon index.js"  
},
```

Входным файлом будет `index.js`. Создайте его. Подключите фреймворк `express`, задайте порт из переменной окружения, либо по умолчанию 8081.

Листинг 48 – Создание сервера

```
server > JS index.js > ...
1  const express = require("express");
2  const app = express();
3  const PORT = process.env.PORT || 8081;
4
5  app.use(express.json());
6
7  app.listen(PORT, () => console.log(`server started on port ${PORT}`));
```

При успешном запуске получите сообщение в терминале.

Листинг 49 – Ответ сервера при успешном запуске

```
sveta@sveta-VirtualBox:~/vs_code/nozzle_1/server$ npm run dev

> server@1.0.0 dev
> nodemon index.js

[nodemon] 2.0.15
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index.js`
server started on port 8081
```

Добавьте URL, по которому будут отправляться данные формы, в компоненте HomePage.vue.

Листинг 50 – Добавление URL для отправки формы

```
<div class="box">
  | | <form action="http://localhost:8081/params"
```

В файле index.js подключите cors для того, чтобы не блокировалась передача данных между сервером и клиентом, подключите модуль body-parser для извлечения данных из тела запроса, данные будут передаваться в формате json.

Листинг 51 – Подключение модулей

```
1  const express = require("express");
2  const cors = require("cors");
3  const bodyParser = require("body-parser");
4  const jsonParser = bodyParser.json();
5
6  const app = express();
7  const PORT = process.env.PORT || 8081;
8
9  const urlencodedParser = bodyParser.urlencoded({ extended: false });
```

Добавьте использование `jsonParser`, `cors`, установите соответствующие заголовки для обмена данными. Из методов разрешите только POST и GET.

Листинг 52 – Добавление заголовков к запросам

```
18 app.use(jsonParser);
19
20 app.use((req, res, next) => {
21   res.setHeader("Access-Control-Allow-Origin", "*");
22   res.setHeader("Access-Control-Allow-Credentials", "true");
23   res.setHeader("Access-Control-Max-Age", "1800");
24   res.setHeader("Access-Control-Allow-Headers", "content-type");
25   res.setHeader("Access-Control-Allow-Methods", "POST, GET");
26   next();
27 });
28 app.use(cors());
```

Создайте переменные, в которых будут храниться входные параметры.

Листинг 53 – Создание переменных

```
11 let p_input = "";
12 let t_input = "";
13 let p_output = "";
14 let g = "";
15 let alpha = "";
16 let betta = "";
```

Функция `app.get()` направляет HTTP GET-запросы по указанному пути. Функция `app.post()` направляет HTTP POST-запросы соответственно. Можно также использовать функцию `app.use()`, которая выполняет все запросы по пути, который начинается с указанного.

По роуту «/params» если не получим никаких данных, то отправим статус 400. Если запрос выполнится успешно, то переменным присваиваются данные из тела запроса `req.body`, и осуществляется переход на страницу с результатами.

Листинг 54 – Создание запроса по отправке входных данных

```
30 app.get("/", function (req, res) {
31     console.log("/");
32 });
33
34 app.post("/params", urlencodedParser, function (req, res) {
35     if (!req.body) return res.sendStatus(400);
36     console.log(req.body);
37     p_input = req.body.p_input;
38     t_input = req.body.t_input;
39     p_output = req.body.p_output;
40     g = req.body.g;
41     alpha = req.body.alpha;
42     betta = req.body.betta;
43     res.status(200);
44     res.redirect("http://localhost:8080/result");
45 });
```

Для GET-запроса просто выведем все данные в виде объекта.

Листинг 55 – GET-запрос по URL «/params»

```
47 app.get("/params", function (req, res) {
48     res.json({
49         p_input: `${p_input}`,
50         t_input: `${t_input}`,
51         p_output: `${p_output}`,
52         g: `${g}`,
53         alpha: `${alpha}`,
54         betta: `${betta}`,
55     });
56 });
```

В браузере нажмите на кнопку «Расчет» с дефолтными значениями параметров. В терминале вы должны увидеть данные, которые отправляются с формы. Также должен произойти переход на форму с результатами.

Листинг 56 – Вывод входных данных в терминал при отправке формы

```
[nodemon] starting `node index.js`
server started on port 8081
[Object: null prototype] {
  p_input: '200000',
  t_input: '1000',
  p_output: '8000',
  g: '2',
  alpha: '14',
  betta: '22'
}
```

Попробуйте ввести другие данные и убедитесь, что они не статические.

Теперь перейдите в файл ResultPage.vue для добавления функции получения данных. В секции script создайте опцию methods (на одном уровне с data, name, components...), добавьте соответствующую функцию.

Для передачи данных будем использовать библиотеку axios. Она представляет собой HTTP-клиент, основанный на промисах и предназначенный для браузеров и для Node.js. После выполнения запроса полученные данные записываются в соответствующие модели и с помощью интерполяции выводятся в шаблон страницы.

Листинг 57 – Функция получения данных от сервера

```
methods: {
  getParams() {
    axios
      .get(`http://localhost:8081/params/`)
      .then((response) => {
        let json = response.data;
        this.P_input = json.p_input;
        this.T_input = json.t_input;
        this.P_output = json.p_output;
        this.G = json.g;
        this.Alpha = json.alpha;
        this.Betta = json.betta;
      })
      .catch((error) => console.log(error));
  },
},
```

Данную библиотеку необходимо импортировать в начало секции script.

Листинг 58 – Импорт библиотеки axios

```
46 <script>
47 import axios from "axios";
```

В хук жизненного цикла created добавьте вызов функции getParams.

Листинг 59 – Вызов функции при создании страницы

```
created() {
  this.getParams();
},
```

Убедитесь, что теперь данные обновляются (Рисунок 9).

Был запущен расчет со следующими входными параметрами:

Давление воздуха P_0 на входе в сопло: **200000** (Па)

Температура воздуха T_0 на входе в сопло: **1000** (K)

Давление воздуха P_1 на выходе из сопла: **8000** (Па)

Расход воздуха G через сопло: **2** (Кг/с)

Угол раскрытия сопла α - входная часть: **15** (°)

Угол раскрытия сопла β - выходная часть: **23** (°)

ОСТАНОВИТЬ РАСЧЕТ

ОТКРЫТЬ PARAVIEW ДЛЯ ПРОСМОТРА ТЕКУЩИХ РЕЗУЛЬТАТОВ

ОТКРЫТЬ ФАЙЛ LOG ДЛЯ ПРОСМОТРА РЕШЕНИЯ

Рисунок 9 – Вывод исходных данных на страницу с результатами

Последовательно нужно вызывать скрипты по чистке предыдущего решения, изменению файла с параметрами, изменению данных файлов OpenFOAM, запуску расчета.

Создадим первую функцию CleanSh, которая вызовет уже созданный bash скрипт с помощью модуля child_process. Данный модуль создает новые дочерние процессы основного процесса Node.js, что позволяет выполнять команды оболочки с их помощью. Node.js является однопоточным, делегирование задач дочернему процессу не блокирует работу приложения.

Есть две функции, которые можно использовать в этих целях: exec и spawn. Основное отличие в том, что первая хранит выходные данные в буфере, они доступны для использования в функции обратного вызова, а результат выполнения второй функции доступен через подписку на события. Поэтому

если данных возвращается не так много, то можно использовать `exec`, функцию `spawn` – при получении большого объема выходной информации.

Подключите модуль `child_process`, также можете создать контроллер, который отвечает за остановку выполнения кода, `signal` необходимо указать в функции `spawn`. Но в данной работе останавливать процессы будем иначе. Просто имейте в виду, что есть и другой вариант.

Листинг 60 – Подключение модуля `child_process`

```
const { exec, spawn } = require("child_process");
let controller = new AbortController();
let { signal } = controller;
```

Подпишитесь на события. В зависимости от успеха выполнения переменной `cleaned` присваивается соответствующее значение.

Листинг 61 – Функция очистки файлов предыдущего решения

```
async function cleanSh() {
  spawnClean = spawn(
    "sh /home/sveta/OpenFOAM/sveta-9/run/nozzle_1/Clean.sh",
    [],
    { shell: true, signal }
  );
  spawnClean.stdout.on("data", (data) => {
    console.log(`stdout: ${data}`);
  });
  spawnClean.stderr.on("data", (data) => {
    console.log(`stderr: ${data}`);
  });
  spawnClean.on("error", (error) => {
    cleaned = "Не удалось очистить файлы предыдущего решения.";
    console.log(`error: ${error.message}`);
  });
  spawnClean.on("spawn", () => {
    cleaned = "Файлы предыдущего решения успешно удалены.";
    console.log(`cleaned`);
  });
  spawnClean.on("close", (code) => {
    console.log(`child process CLEAN exited with code ${code}`);
  });
}
```

Объявите используемые переменные.

Листинг 62 – Создание переменных

```
let cleaned = "";  
let spawnClean = null;
```

Теперь добавьте вызов данной функции в `app.post` после перехода на страницу с результатами, заключите ее в блок `try – catch`.

Листинг 63 – Вызов функции CleanSh

```
36 app.post("/params", urlencodedParser, function (req, res) {  
37     if (!req.body) return res.sendStatus(400);  
38     console.log(req.body);  
39     p_input = req.body.p_input;  
40     t_input = req.body.t_input;  
41     p_output = req.body.p_output;  
42     g = req.body.g;  
43     alpha = req.body.alpha;  
44     betta = req.body.betta;  
45     res.status(200);  
46     res.redirect("http://localhost:8080/result");  
47     try {  
48         cleanSh();  
49     } catch {  
50         res.send("Не удалось запустить расчет.");  
51     }  
52 });
```

Попробуйте запустить расчет, в терминале должен отобразиться результат выполнения `bash` скрипта.

Листинг 64 – Результат выполнения bash скрипта

```
cleaned  
stderr: rm: невозможно удалить 'log'  
stderr: : Нет такого файла или каталога  
  
stdout: Done  
  
child process CLEAN exited with code 0
```

Добавьте использование роута `«/cleaned»` для отправки и получения информации о выполнении скрипта `Clean.sh` в формате `json`.

Листинг 65 – Использование роута «/cleaned»

```
65 app.use("/cleaned", function (req, res, next) {  
66     res.send({ cleaned: `${cleaned}` });  
67     next();  
68 });
```

С помощью программы Postman (установите ее через терминал на виртуальную машину) проверьте ответ сервера. Перейдите на соответствующий URL (Рисунок 10).

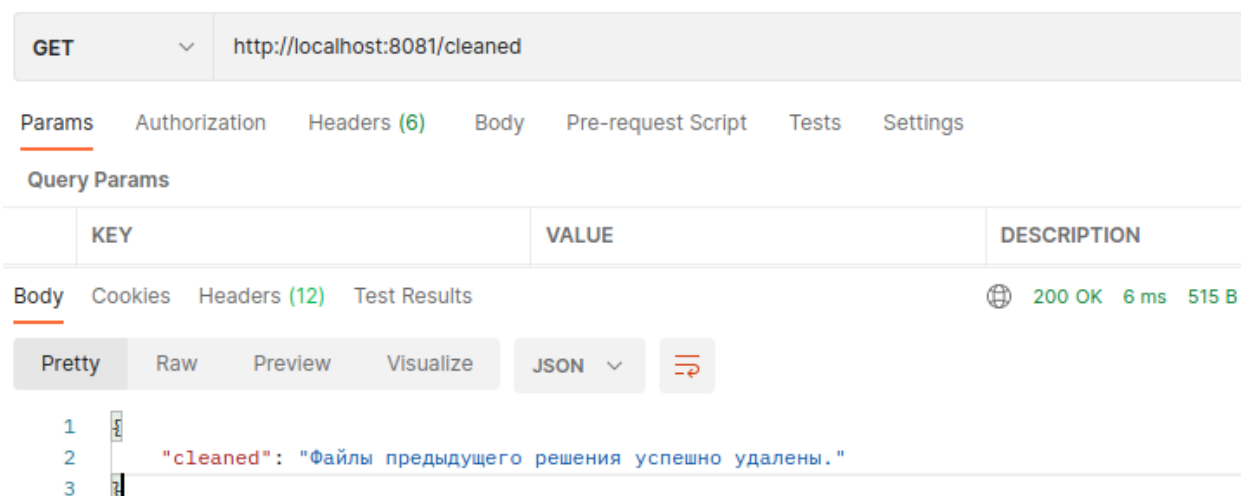


Рисунок 10 – Результат выполнения функции cleanSh

Измените файл ResultPage.vue. В блок с классом rightside добавьте заголовок <h2> и параграф <p>, который будет динамически изменять свой класс в зависимости от условия. В интерполяции выводим значение переменной cleaned либо статический текст, если переменной еще не было присвоено значение.

Листинг 66 – Вывод информации о выполнении функции cleanSh

```
<div class="rightside">
  <h2>Подготовка к расчету:</h2>
  <p
    :class="{
      green: cleaned == 'Файлы предыдущего решения успешно удалены.',
      red: cleaned == 'Не удалось очистить файлы предыдущего решения.',
    }"
  >
    {{ cleaned || "Очистка файлов предыдущего решения" }}
  </p>
</div>
```

Добавьте модель cleaned со значением пустой строки в свойство data компонента ResultPage.vue.

Листинг 67 – Добавление модели cleaned

```
data() {  
  return {  
    P_input: "",  
    T_input: "",  
    P_output: "",  
    G: "",  
    Alpha: "",  
    Betta: "",  
    cleaned: "",  
  }  
},
```

Добавьте метод `clean`, который делает запрос на URL «/cleaned», полученный ответ присваивает переменной `cleaned`.

Листинг 68 – Функция получения ответа о результате выполнения `cleanSh`

```
clean() {  
  axios  
    .get(`http://localhost:8081/cleaned/`)  
    .then((response) => {  
      let json = response.data;  
      this.cleaned = json.cleaned;  
      console.log(json);  
    })  
    .catch((error) => console.log(error));  
},
```

Добавьте его вызов после получения параметров в функцию `getParams`.

Листинг 69 – Вызов функции `clean` после получения параметров

```
getParams() {  
  axios  
    .get(`http://localhost:8081/params/`)  
    .then((response) => {  
      let json = response.data;  
      this.P_input = json.p_input;  
      this.T_input = json.t_input;  
      this.P_output = json.p_output;  
      this.G = json.g;  
      this.Alpha = json.alpha;  
      this.Betta = json.betta;  
      this.clean();  
    })  
},
```

Создайте стили для классов `green` и `red`.

Листинг 70 – Стили для классов red и green

```
.green {  
  color: ■green;  
}  
.red {  
  color: ■rgb(163, 27, 27);  
}
```

Проверьте, что на странице с результатами теперь отображается информация о выполнении скрипта по очистке файлов предыдущего решения (Рисунок 11).

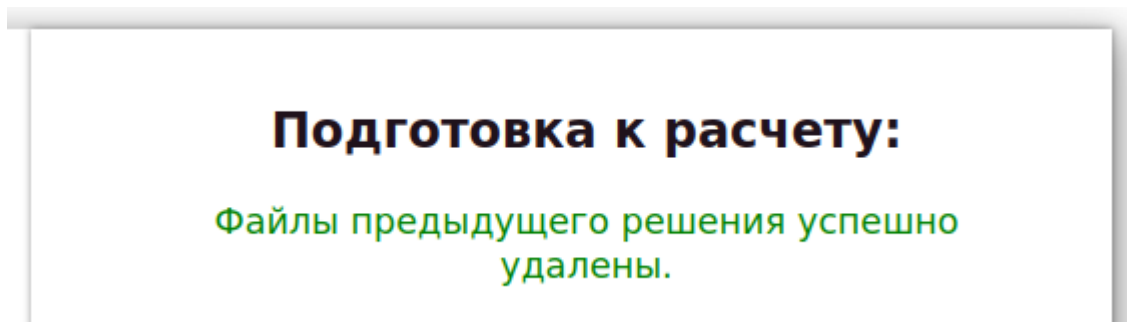


Рисунок 11 – Отображение результата выполнения функции на клиенте

По аналогии создайте функцию по изменению файла с входными параметрами. Команда для изменения данных sed должна быть написана в одну строку.

Листинг 71 – Функция changeUserParams

```
async function changeUserParams() {  
  spawnchangeUserParams = spawn(  
    `cd /home/sveta/OpenFOAM/sveta-9/run/nozzle_1/;  
    sed -i "2s/.*/${p_input};;5s/.*/${t_input};;8s/.*/${p_output};;11s/.*/  
    /${g};;14s/.*/${alpha};;17s/.*/${beta}/" params.txt;  
    `,  
    [],  
    { shell: true, signal }  
  );  
  spawnchangeUserParams.stdout.on("data", (data) => {  
    //console.log(`stdout: ${data}`);  
  });  
  spawnchangeUserParams.stderr.on("data", (data) => {  
    console.log(`stderr: ${data}`);  
  });  
}
```

Листинг 72 - Функция changeUserParams продолжение

```
spawnchangeUserParams.on("error", (error) => {
  changed_params =
    "Не удалось внести изменения в файл с входными значениями.";
  console.log(`error: ${error.message}`);
});
spawnchangeUserParams.on("spawn", () => {
  changed_params = "Файл с входными значениями успешно изменен.";
  console.log(`params.txt changed`);
});
spawnchangeUserParams.on("close", (code) => {
  console.log(`child process changeUserParams exited with code ${code}`);
});
}
```

Добавьте в предыдущую функцию очистки файлов после ее успешного выполнения вызов функции по изменению параметров.

Листинг 73 – Вызов функции изменения файла с входными параметрами

```
spawnClean.on("spawn", () => {
  cleaned = "Файлы предыдущего решения успешно удалены.";
  console.log(`cleaned`);
  changeUserParams();
});
```

Создайте соответствующие переменные.

Листинг 74 – Создание переменных

```
let changed_params = "";
let spawnchangeUserParams = null;
```

Добавьте использование роута «/changeUserParams» для отправки и получения данных о выполнении функции.

Листинг 75 – Использование роута «/changeUserParams»

```
app.use("/changeUserParams", function (req, res, next) {
  res.send({ changed: `${changed_params}` });
  next();
});
```

Проверьте с помощью программы Postman, что данные отправляются по пути «/changeUserParams» (Рисунок 12).

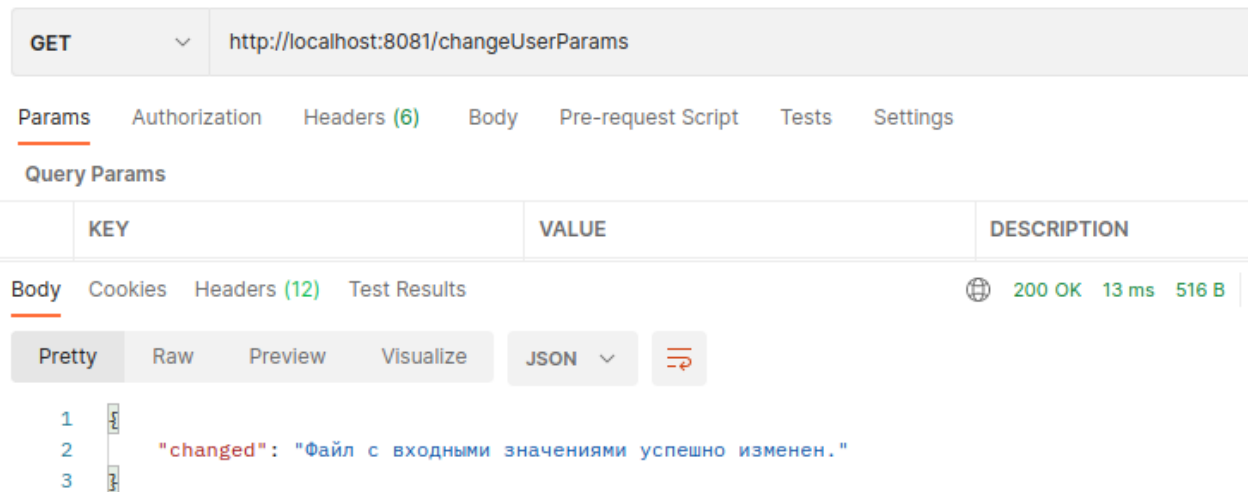


Рисунок 12 – Результат выполнения функции `changeUserParams`

В файл `ResultPage.vue` добавьте функцию для получения данных по роуту `«/changeUserParams»`.

Листинг 76 – Функция получения ответа о результате выполнения `changeUserParams`

```
changeParams() {
  axios
    .get(`http://localhost:8081/changeUserParams/`)
    .then((response) => {
      let json = response.data;
      this.changeUserParams = json.changed;
      console.log(json);
    })
    .catch((error) => console.log(error));
},
```

В функцию `clean` добавьте вызов созданной функции.

Листинг 77 – Вызов функции `changeParams` после очистки файлов предыдущего решения

```
clean() {
  axios
    .get(`http://localhost:8081/cleaned/`)
    .then((response) => {
      let json = response.data;
      this.cleaned = json.cleaned;
      console.log(json);
      this.changeParams();
    })
    .catch((error) => console.log(error));
},
```

Создайте новую переменную.

Листинг 78 – Создание переменной

```
changeUserParams: "",
```

В шаблоне страницы создайте новый параграф, который по умолчанию будет выводить строку «Изменение файла с входными значениями», но в зависимости от ответа сервера динамически изменит и текст, и цвет параграфа.

Листинг 79 – Вывод информации клиенту о результате выполнения функции changeParams

```
<p
  :class="{
    green:
      changeUserParams ==
        'Файл с входными значениями успешно изменен.',
    red:
      changeUserParams ==
        'Не удалось внести изменения в файл с входными значениями.',
  }"
>
  {{ changeUserParams || "Изменение файла с входными данными" }}
</p>
```

Проверьте, что все корректно отображается в браузере (Рисунок 13).

Подготовка к расчету:

Файлы предыдущего решения успешно
удалены.
Файл с входными значениями успешно изменен.

Рисунок 13 – Отображение результата выполнения функции на клиенте

Теперь создайте функцию по изменению файлов OpenFOAM. Необходимо вызвать уже написанный скрипт ChangeParams.sh. По аналогии подпишитесь на события вывода данных, обнаружения ошибок, успешного выполнения и завершения процесса.

Листинг 80 – Функция changeParamsSh

```
async function changeParamsSh() {
  spawnchangeParams = spawn(
    "sh /home/sveta/OpenFOAM/sveta-9/run/nozzle_1/ChangeParams.sh",
    [],
    { shell: true, signal }
  );
  spawnchangeParams.stdout.on("data", (data) => {
    console.log(`stdout: ${data}`);
  });
  spawnchangeParams.stderr.on("data", (data) => {
    console.log(`stderr: ${data}`);
  });
  spawnchangeParams.on("error", (error) => {
    changed_files = "Не удалось изменить параметры в файлах OpenFOAM.";
    console.log(`error: ${error.message}`);
  });
  spawnchangeParams.on("spawn", () => {
    changed_files = "Файлы OpenFOAM успешно изменены.";
    console.log(`OpenFOAM changed`);
  });
  spawnchangeParams.on("close", (code) => {
    console.log(`child process changeParamsSh exited with code ${code}`);
  });
}
```

В созданную ранее функцию по изменению файла params.txt добавьте вызов следующей функции.

Листинг 81 – Вызов функции changeParamsSh

```
spawnchangeUserParams.on("spawn", () => {
  changed_params = "Файл с входными значениями успешно изменен.";
  console.log(`params.txt changed`);
  changeParamsSh();
});
```

Создайте соответствующие переменные.

Листинг 82 – Создание переменных

```
let changed_files = "";
let spawnchangeParams = null;
```

Создайте роут по обработке данного запроса.

Листинг 83 – Использование роута «/changeParamsSh»

```
app.use("/changeParamsSh", function (req, res, next) {
  res.send({ changed_files: `${changed_files}` });
  next();
});
```

Проверьте в программе Postman корректность выполнения функции (Рисунок 14).

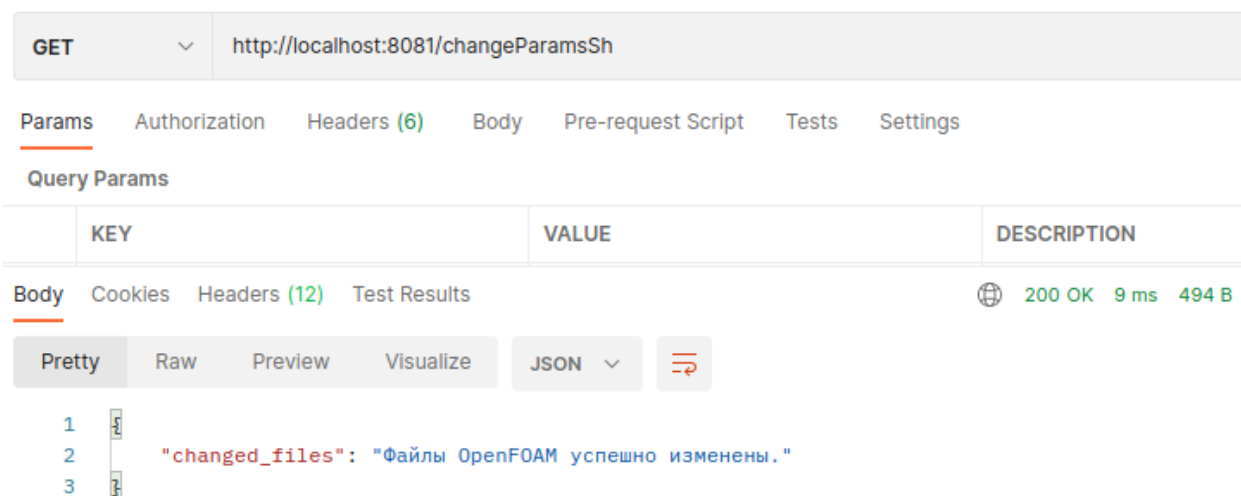


Рисунок 14 – Результат выполнения функции `changeParamsSh`

На клиенте добавьте функцию по выполнению GET-запроса по созданному роуту.

Листинг 84 – Функция `changeFiles`

```
changeFiles() {  
  axios  
    .get(`http://localhost:8081/changeParamsSh/`)  
    .then((response) => {  
      let json = response.data;  
      this.changed_files = json.changed_files;  
      console.log(json);  
    })  
    .catch((error) => console.log(error));  
},
```

В функцию `changeParams` добавьте вызов данной функции.

Листинг 85 – Вызов функции `changeFiles`

```
changeParams() {  
  axios  
    .get(`http://localhost:8081/changeUserParams/`)  
    .then((response) => {  
      let json = response.data;  
      this.changeUserParams = json.changed;  
      console.log(json);  
      this.changeFiles();  
    })  
    .catch((error) => console.log(error));  
},
```

Объявите новую переменную.

Листинг 86 – Создание переменной

```
changed_files: "",
```

В шаблон страницы добавьте новый параграф с выводом информации об изменении файлов OpenFOAM.

Листинг 87 – Отображение результата выполнения функции изменения файлов OpenFOAM на клиенте

```
<p
  :class="{
    green: changed_files == 'Файлы OpenFOAM успешно изменены.',
    red:
      changed_files ==
        'Не удалось изменить параметры в файлах OpenFOAM.',
  }"
>
  {{ changed_files || "Изменение файлов OpenFOAM" }}
</p>
```

Проверьте корректность выполнения функции в браузере (Рисунок 15).

Подготовка к расчету:

Файлы предыдущего решения успешно
удалены.
Файл с входными значениями успешно изменен.
Файлы OpenFOAM успешно изменены.

Рисунок 15 – Результат выполнения функции

Создайте следующую функцию, которая вызывает скрипт Run.sh. По аналогии подпишитесь на события вывода данных, обнаружения ошибок, успешного выполнения и завершения. Присвойте значение переменной runned в зависимости от результата выполнения.

Листинг 88 – Функция runSh

```
async function runSh() {
  spawnRun = spawn("sh /home/sveta/OpenFOAM/sveta-9/run/nozzle_1/Run.sh", [], {
    shell: true,
    signal,
  });

  spawnRun.stdout.on("data", (data) => {
    //console.log(`stdout: ${data}`);
  });

  spawnRun.stderr.on("data", (data) => {
    console.log(`stderr run: ${data}`);
  });

  spawnRun.on("error", (error) => {
    runned = "Ошибка при выполнении расчета";
    console.log(`error: ${error.message}`);
  });

  spawnRun.on("spawn", () => {
    runned = "Расчет успешно запущен. Ожидайте...";
    console.log(`runned calc`);
  });

  spawnRun.on("close", (code) => {
    console.log(`child process runSh exited with code ${code}`);
  });
}
```

В созданную ранее функцию по изменению файлов OpenFOAM добавьте вызов функции runSh.

Листинг 89 – Вызов функции runSh

```
spawnchangeParams.on("spawn", () => {
  changed_files = "Файлы OpenFOAM успешно изменены.";
  console.log(`OpenFOAM changed`);
  runSh();
});
```

Откройте скрипт Run.sh и оставьте строки только по созданию сетки, остальные закомментируйте. Это необходимо на данном этапе для того, чтобы не начинать выполнение расчета и других операций, не нагружать процессор. Сейчас будем отслеживать только создание сетки.

Листинг 90 – Изменение файла Run.sh

```
cd ~/SALOME-9.7.0-native-UB20.04-SRC
./salome -t /home/sveta/OpenFOAM/sveta-9/run/nozzle_1/geometry/Mesh.py

cd /home/sveta/OpenFOAM/sveta-9/run/nozzle_1
ideasUnvToFoam geometry/Mesh.unv
transformPoints "scale = (0.001 0.001 0.001)"
sed -i '33s/patch/wall/;27s/patch/wedge/;57s/patch/wedge/' constant/polyMesh/-
boundary

#width=$(head -n 20 params.txt | tail -n 1)
#height=$(head -n 23 params.txt | tail -n 1)

#sed -i "26s/.*/      box (-${width} 0 -1) (0 ${height} 1);/" system/-
setFieldsDict;
#setFields
#decomposePar
#mpirun -np 4 rhoPimpleFoam -parallel >log
#reconstructPar
#paraFoam
```

Объявите новые переменные, присвойте им начальные значения.

Листинг 91 – Создание переменных

```
let runned = "";
let spawnRun = null;
```

Добавьте роут по выполнению запросов.

Листинг 92 – Использование роута «/runned»

```
app.use("/runned", function (req, res, next) {
  res.send({ runned: `${runned}` });
  next();
});
```

Проверьте корректность работы функции в Postman (Рисунок 16).

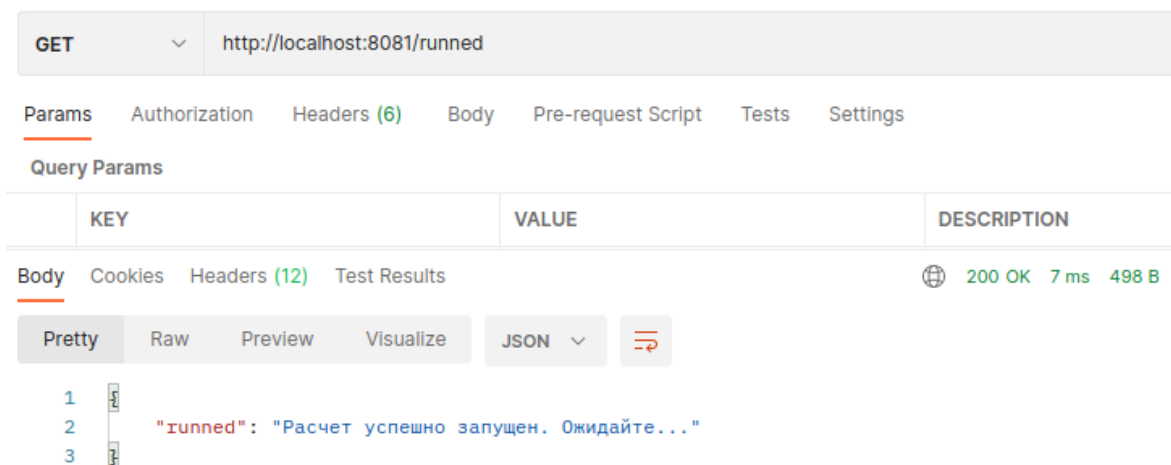


Рисунок 16 – Результат выполнения функции runSh

На клиенте создайте функцию по получению данных о запуске скрипта Run.sh.

Листинг 93 – Функция run

```
run() {  
  axios  
    .get(`http://localhost:8081/runned/`)  
    .then((response) => {  
      let json = response.data;  
      this.runned = json.runned;  
      console.log(json);  
    })  
    .catch((error) => console.log(error));  
},
```

Добавьте ее выполнение в функцию changeFiles.

Листинг 94 – Вызов функции run

```
changeFiles() {  
  axios  
    .get(`http://localhost:8081/changeParamsSh/`)  
    .then((response) => {  
      let json = response.data;  
      this.changed_files = json.changed_files;  
      console.log(json);  
      this.run();  
    })  
}
```

Объявите новую переменную.

Листинг 95 – Создание переменной

```
runned: "",
```

В шаблон добавьте еще один параграф.

Листинг 96 – Отображение информации о результате выполнения функции

```
<p  
  :class="{  
    green: runned == 'Расчет успешно запущен. Ожидайте...',  
    red: runned == 'Ошибка при выполнении расчета',  
  }"  
>  
  {{ runned || "Запуск расчета" }}  
</p>
```

Проверьте работу приложения в браузере (Рисунок 17).

Подготовка к расчету:

Файлы предыдущего решения успешно удалены.

Файл с входными значениями успешно изменен.

Файлы OpenFOAM успешно изменены.

Расчет успешно запущен. Ожидайте...

Рисунок 17 – Результат выполнения функции

Следующее сообщение, которое необходимо выводить на форму – создание сетки. Ведь если сетка не будет построена из-за некорректных данных, то пользователь должен получить об этом сообщение, а расчет завершиться.

На сервере создаем функцию, которая будет отслеживать через определенные промежутки времени наличие файла Mesh.unv в каталоге geometry. Выполнять данный функционал будем с помощью модуля fs и его функции existsSync, результат запишем в переменную existsMesh. Если данная переменная возвращает true, значит сетка успешно построилась, очистим таймаут, иначе вызываем снова данную функцию через 4 секунды (можете задать свой интервал). В логах выведем сообщение о наличии файла на данной итерации.

Листинг 97 – Функция createMesh

```
function createMesh() {  
  existsMesh = "";  
  try {  
    existsMesh = fs.existsSync(  
      "/home/sveta/OpenFOAM/sveta-9/run/nozzle_1/geometry/Mesh.unv"  
    );  
    console.log("Mesh exists: ", existsMesh);  
  } catch (e) {  
    meshCreated = "Не удалось создать сетку";  
    console.log(e);  
  }  
  if (existsMesh) {  
    meshCreated = "Сетка успешно создана";  
    clearTimeout(id);  
  } else setTimeout(createMesh, 4000);  
}
```

После запуска расчета запустим таймаут, который вызовет функцию createMesh через 4 секунды.

Листинг 98 – Вызов функции createMesh

```
spawnRun.on("spawn", () => {  
  runned = "Расчет успешно запущен. Ожидайте...";  
  console.log(`runned calc`);  
  id = setTimeout(createMesh, 4000);  
});
```

Подключите модуль fs и объявите переменные.

Листинг 99 – Подключение модуля и объявление переменных

```
const fs = require("fs");  
let existsMesh = false;  
let meshCreated = "";  
let id = "";
```

Задайте роут по выполнению запроса о создании сетки.

Листинг 100 – Использование роута «/meshCreated»

```
app.use("/meshCreated", function (req, res, next) {  
  res.send({ meshCreated: `${meshCreated}` });  
  next();  
});
```

Сохраните изменения, запустите расчет. Откройте каталог geometry и убедитесь, что на этапе очистки решения файл Mesh.unv был удален, а на этапе выполнения скрипта Run.sh снова создан. В терминал должна выводиться информация о том, на какой итерации сетка была построена.

Листинг 101 – Вывод результата работы функции createMesh

```
Mesh exists: false  
Mesh exists: true
```

Проверьте с помощью программы Postman, что данные успешно отправляются (Рисунок 18).

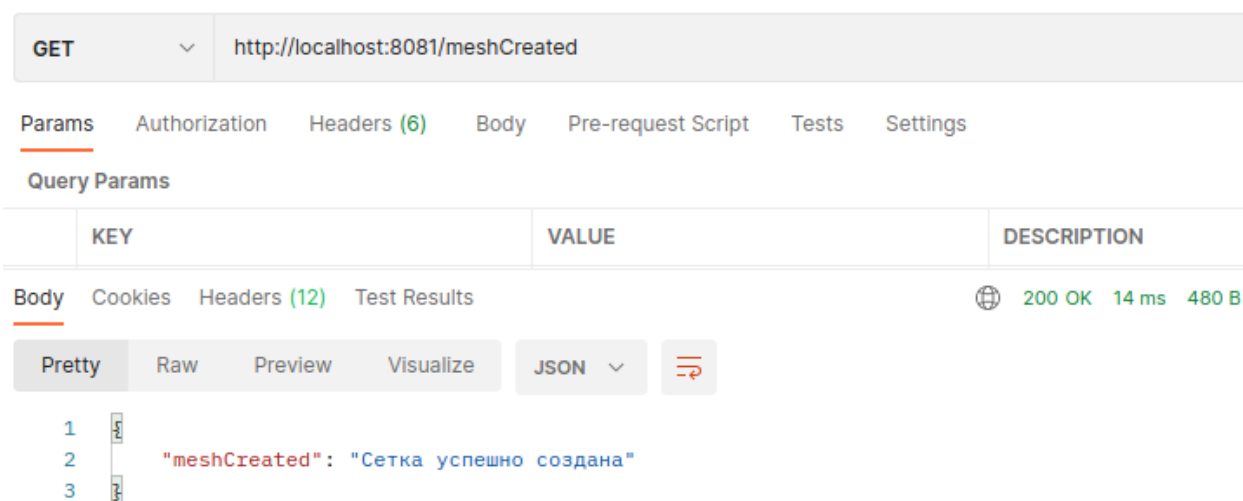


Рисунок 18 – Результат работы функции createMesh

Теперь попробуйте увеличить давление на входе в 10 раз (давление должно быть 2000000 Па), остальные параметры не изменяйте. В данных условиях сетка не будет построена. Ошибка выведется в терминал.

Листинг 102 – Ошибка построения сетки

```
Mesh exists: false
stderr run: Traceback (most recent call last):
  File "/home/sveta/OpenFOAM/sveta-9/run/nozzle_1/geometry/Mesh.py", line 451, in <module>
    Revolution_1_1, Group_1_1, Group_2_1, Group_3_1, Group_4_1, Group_5_1, Group_6_1, Group_7_1, Group_8_1, Group_9_1, inlet, front, wall, outletLeft, outletTop, outletRight, back, = SHAPERSTUDY.shape(model.featureStringId(Revolution_1))
TypeError: cannot unpack non-iterable NoneType object

stderr run: ERROR:salomeContext:SystemExit 1 in method _runAppli.
```

Остановите сервер (Ctrl + C в терминале).

Создайте обработчик данной ошибки. Как можно было заметить, в выводе сообщения об ошибке присутствует строка: ERROR:salome. Она говорит об ошибке во время выполнения скрипта для Salome. При ошибке в решении OpenFOAM сообщение будет содержать строку: Foam::error. Данную ошибку обрабатываем позже.

В случае возникновения ошибки при построении сетки переменной meshCreatedAbort присваивается значение true и вызывается функция createMesh.

Листинг 103 – Обработка ошибки построения сетки

```
spawnRun.stderr.on("data", (data) => {
  console.log(`stderr run: ${data}`);
  if (
    data.includes(
      'ERROR:salome'
    )
  ) {
    meshCreatedAbort = true;
    createMesh();
  }
});
```

В функцию createMesh добавьте еще одно условие, которое будет выполняться при значении переменной meshCreatedAbort, равной true. Переменная meshCreated, отвечающая за ответ от сервера о создание сетки, принимает значение «Сетка не может быть построена с данными входными параметрами». Останавливается таймаут.

Листинг 104 – Изменение значения переменной при возникновении ошибки

```
}else if (meshCreatedAbort) {
  meshCreated =
    "Сетка не может быть построена с данными входными параметрами.";
  console.log(meshCreated);
  clearTimeout(id);
} else setTimeout(createMesh, 4000);
```

Объявите переменную.

Листинг 105 – Создание переменной

```
let meshCreatedAbort = false;
```

Запустите снова сервер, введите давление на входе 2000000, вывод в терминал должен быть следующим.

Листинг 106 – Вывод в терминал обработанной ошибки

```
stderr run: ERROR:salomeContext:SystemExit 1 in method _runAp
pli.

Mesh exists: false
Сетка не может быть построена с данными входными параметрами.
```

Теперь выведем информацию о сетке на страницу. Создайте функцию createdMesh, которая выполняет GET-запрос по URL «/meshCreated». Если

значение meshCreated не равно пустой строке, то таймаут останавливается, иначе функция проверки создания сетки вызовется вновь через 4 секунды.

Листинг 107 – Функция createdMesh

```
createdMesh() {  
  axios  
    .get(`http://localhost:8081/meshCreated/`)  
    .then((response) => {  
      let json = response.data;  
      this.meshCreated = json.meshCreated;  
      if (json.meshCreated !== "") {  
        clearTimeout(this.id);  
      } else {  
        setTimeout(this.createdMesh, 4000);  
      }  
      console.log(json);  
    })  
    .catch((error) => console.log(error));  
},
```

После успешного запуска файла Run.sh добавьте таймаут, который вызовет функцию createMesh.

Листинг 108 – Вызов функции createdMesh

```
run() {  
  axios  
    .get(`http://localhost:8081/runned/`)  
    .then((response) => {  
      let json = response.data;  
      this.runned = json.runned;  
      console.log(json);  
      if (this.runned == "Расчет успешно запущен. Ожидайте...")  
        this.id = setTimeout(this.createdMesh, 4000);  
    })  
    .catch((error) => console.log(error));  
},
```

Объявите переменные.

Листинг 109 – Создание переменных

```
meshCreated: "",  
id: "",
```

В шаблон приложения добавьте новый параграф, который выводит результат построения сетки. Также динамически присваивается класс и текст в зависимости от ответа сервера.

Листинг 110 – Вывод информации о построении сетки

```
<p
  :class="{
    green: meshCreated == 'Сетка успешно создана',
    red:
      meshCreated ==
        'Сетка не может быть построена с данными входными параметрами.' ||
        meshCreated ==
        'Не удалось создать сетку',
  }"
>
  {{ meshCreated || "Создание сетки" }}
</p>
```

Создадим функцию `clearAllPathAndVars` на сервере, в которой будем сбрасывать значения всех переменных.

Листинг 111 – Функция `clearAllPathAndVars`

```
function clearAllPathAndVars() {
  existsMesh = false;
  meshCreated = "";
  id = "";
  p_input = "";
  t_input = "";
  p_output = "";
  g = "";
  alpha = "";
  betta = "";
  cleaned = "";
  spawnClean = null;
  changed_params = "";
  spawnchangeUserParams = null;
  changed_files = "";
  spawnchangeParams = null;
  runned = "";
  spawnRun = null;
  meshCreatedAbort = false;
  console.log("clearAllPathAndVars");
}
```

Добавьте в `app.post` после перехода на страницу с результатами выполнение функции `clearAllPathAndVars`.

Листинг 112 – Вызов функции `clearAllPathAndVars`

```
res.redirect("http://localhost:8080/result");
clearAllPathAndVars();
```

Проверьте работу приложения в браузере (Рисунок 19 - Рисунок 20).

**Был запущен расчет со
следующими входными
параметрами:**

Давление воздуха P_0 на входе в сопло: **2000000** (Па)
Температура воздуха T_0 на входе в сопло: **1000** (K)
Давление воздуха P_1 на выходе из сопла: **8000** (Па)
Расход воздуха G через сопло: **2** (Kg/c)
Угол раскрытия сопла α - входная часть: **14** (°)
Угол раскрытия сопла β - выходная часть: **22** (°)

Подготовка к расчету:

Файлы предыдущего решения успешно удалены.
Файл с входными значениями успешно изменен.
Файлы OpenFOAM успешно изменены.
Расчет успешно запущен. Ожидайте...
Сетка не может быть построена с данными входными параметрами.

Рисунок 19 – Ошибка при построении сетки

**Был запущен расчет со
следующими входными
параметрами:**

Давление воздуха P_0 на входе в сопло: **200000** (Па)
Температура воздуха T_0 на входе в сопло: **1000** (K)
Давление воздуха P_1 на выходе из сопла: **8000** (Па)
Расход воздуха G через сопло: **2** (Kg/c)
Угол раскрытия сопла α - входная часть: **14** (°)
Угол раскрытия сопла β - выходная часть: **22** (°)

Подготовка к расчету:

Файлы предыдущего решения успешно удалены.
Файл с входными значениями успешно изменен.
Файлы OpenFOAM успешно изменены.
Расчет успешно запущен. Ожидайте...
Сетка успешно создана

Рисунок 20 – Сетка успешно создана

По аналогии создаем на сервере функцию проверки распараллеливания расчета. Если создался каталог processor0, то и остальные должны создаваться успешно. Можете вместо processor0 написать последний каталог – processor3 (если выделили 4 ядра).

Листинг 113 – Функция checkProcessors

```
function checkProcessors() {  
  existsProcessors = "";  
  try {  
    existsProcessors = fs.existsSync(  
      "/home/sveta/OpenFOAM/sveta-9/run/nozzle_1/processor0"  
    );  
    console.log("Exists_processors: ", existsProcessors);  
  } catch (e) {  
    processorsCreated = "Не удалось распараллелить задачу"  
    console.log(e);  
  }  
  if (existsProcessors) {  
    processorsCreated = "Задача успешно распараллелена"  
    clearTimeout(id_processors);  
  } else  
    setTimeout(checkProcessors, 4000);  
}
```

В функцию createMesh после успешного создания сетки добавьте таймаут. Время вызова можете изменить в зависимости от мощности компьютера.

Листинг 114 – Вызов функции checkProcessors

```
if (existsMesh) {  
  meshCreated = "Сетка успешно создана";  
  clearTimeout(id);  
  id_processors = setTimeout(checkProcessors, 4000);  
}
```

Объявите переменные.

Листинг 115 – Создание переменных

```
let existsProcessors = false;  
let id_processors = "";  
let processorsCreated = "";
```

В функцию clearAllPathAndVars добавьте сброс значений созданным переменным.

Листинг 116 – Сброс значений к дефолтным

```
existsProcessors = false;  
id_processors = "";  
processorsCreated = "";  
console.log("clearAllPathAndVars");  
}
```

Задайте роут по выполнению запроса проверки распараллеливания задачи.

Листинг 117 – Использование роута «/checkProcessors»

```
app.use("/checkProcessors", function (req, res, next) {  
  res.send({  
    existsProcessors: `${processorsCreated}`,  
  });  
  next();  
});
```

Раскомментируйте строку decomposePar в файле Run.sh.

Листинг 118 – Изменение файла Run.sh

```
16 setFields  
17 decomposePar  
18 #mpirun -np 4 rhoPimpleFoam -parallel >log  
19 #reconstructPar  
20 #paraFoam
```

Нажмите на кнопку расчет, в терминал теперь должна выводиться информация не только о создании сетки, но и о распараллеливании задачи.

Листинг 119 – Вывод информации о распараллеливании задачи

```
Mesh exists: false
Mesh exists: true
Exists_processors: false
Exists processors: true
```

Проверьте корректность отправки данных в Postman (Рисунок 21).

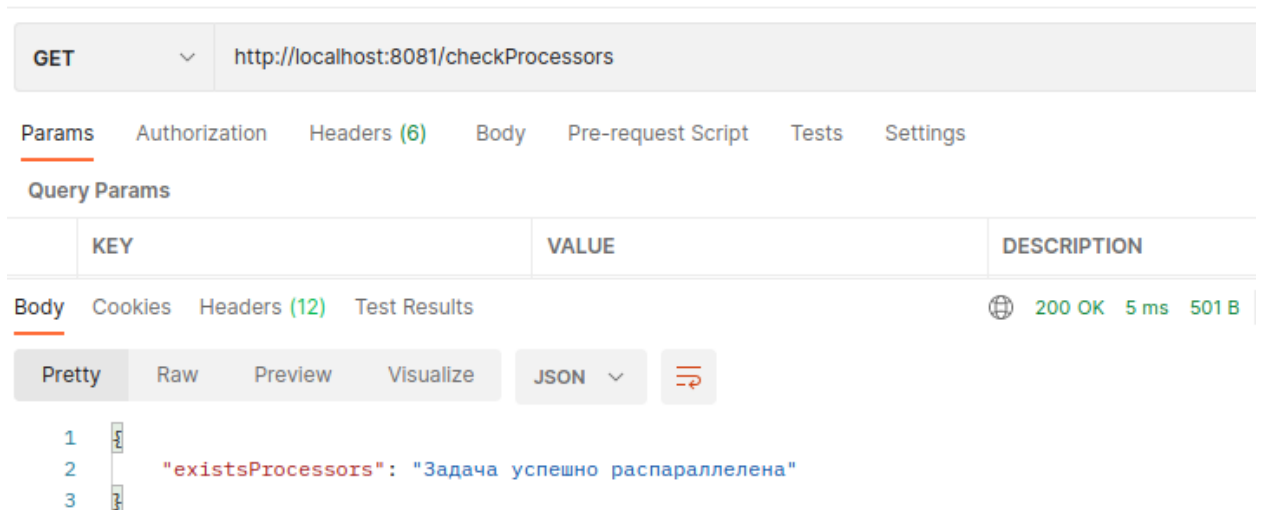


Рисунок 21 – Результат выполнения функции checkProcessors

На клиенте создайте функцию по отслеживанию распараллеливания расчета. В случае если existProcessors не пустая строка, то функция вызываться больше не будет, иначе задается таймаут.

Листинг 120 – Функция createdProcessors

```
createdProcessors() {
  axios
    .get(`http://localhost:8081/checkProcessors/`)
    .then((response) => {
      let json = response.data;
      this.checkProcessors = json.existsProcessors;
      if (json.existsProcessors !== "") {
        clearTimeout(this.id_processor);
      } else {
        setTimeout(this.createdProcessors, 4000);
      }
      console.log(json);
    })
    .catch((error) => console.log(error));
},
```

В функцию `createdMesh` добавьте таймаут по вызову созданной функции.

Листинг 121 – Вызов функции `createdProcessors`

```
if (json.meshCreated !== "") {  
    clearTimeout(this.id);  
    this.id_processor = setTimeout(this.createdProcessors, 4000);  
}
```

Объявите переменные.

Листинг 122 – Создание переменных

```
checkProcessors: "",  
id_processor: "",
```

В шаблон добавьте новый параграф, содержащий информацию о распараллеливании задачи.

Листинг 123 – Вывод информации клиенту о распараллеливании задачи

```
<p  
  :class="{  
    green: checkProcessors == 'Задача успешно распараллелена',  
    red: checkProcessors == 'Не удалось распараллелить задачу',  
  }"  
>  
  {{ checkProcessors || "Распараллеливание задачи" }}  
</p>
```

Проверьте работу приложения в браузере (Рисунок 22).

Подготовка к расчету:

Файлы предыдущего решения успешно удалены.

Файл с входными значениями успешно изменен.

Файлы OpenFOAM успешно изменены.

Расчет успешно запущен. Ожидайте...

Сетка успешно создана

Задача успешно распараллелена

Рисунок 22 – Результат выполнения функции

На данном этапе расчет может быть начат, поэтому необходимо создать функцию, которая будет отслеживать время выполненного расчета, то есть

созданные каталоги в processor*. Назовем ее checkResults. Выполнять проверку можно в каталоге любого из процессоров. Далее следует указывать имя папки. Результаты записываются с интервалов в 0.001 с, поэтому они будут иметь имена с таким шагом. Для этого используем переменную folderTime. На каждой итерации обеспечим вывод в консоль сообщения о наличии или отсутствии решения для конкретного времени. Если решение будет обнаружено, то в массив existsResult с помощью метода push добавится строка, значение folderTime изменится на следующее и пока оно не достигнет времени окончания, данная функция будет вызываться снова. Время окончания установите в значение 0.5, также убедитесь, что данное значение установлено в файле controlDict.

Листинг 124 – Функция checkResults

```
function checkResults() {
  try {
    existsFolderResults = fs.existsSync(
      `/home/sveta/OpenFOAM/sveta-9/run/nozzle_1/processor0/${folderTime}`
    );
    console.log("Exists_results: ", existsFolderResults, folderTime);
  } catch (e) {
    console.log(existsResults);
    console.log(e);
  }
  if (existsFolderResults) {
    existsResults.push(`Решение готово для времени ${folderTime}`);
    clearTimeout(id_results);
    folderTime += 0.0001;
    folderTime = +folderTime.toFixed(4);
    if (folderTime < 0.5) {
      id_results = setTimeout(checkResults, 10000);
    }
  } else setTimeout(checkResults, 10000);
}
```

Добавьте установку таймаута после распараллеливания задачи.

Интервал можете изменить.

Листинг 125 – Вызов функции checkResults

```
if (existsProcessors) {
  processorsCreated = "Задача успешно распараллелена"
  clearTimeout(id_processors);
  id_results = setTimeout(checkResults, 10000);
}
```

Объявите переменные.

Листинг 126 – Создание переменных

```
let existsFolderResults = false;
let existsResults = [];
let folderTime = 0.0001;
let id_results = "";
```

Добавьте сброс значений в функцию clearAllPathAndVars.

Листинг 127 – Сброс значений к дефолтным

```
existsFolderResults = false;
existsResults = [];
folderTime = 0.0001;
id_results = "";
console.log("clearAllPathAndVars");
}
```

Раскомментируйте строку, отвечающую за запуск расчета.

Листинг 128 – Изменение файла Run.sh

```
17 decomposePar
18 mpirun -np 4 rhoPimpleFoam -parallel >log
19 #reconstructPar
20 #paraFoam
```

Добавьте роут для запросов о ходе выполнения расчета.

Листинг 129 – Использование роута «/checkResults»

```
app.use("/checkResults", function (req, res, next) {
  res.send({ existsFolderResults: `${existsResults}` });
  next();
});
```

В зависимости от мощности компьютера и заданного интервала результаты могут принимать значение false.

Листинг 130 – Вывод в терминал сообщений о готовых решениях

```
Mesh exists: false
Mesh exists: true
Exists_processors: true
Exists_results: false 0.0001
Exists_results: true 0.0001
Exists_results: true 0.0002
Exists_results: true 0.0003
Exists_results: true 0.0004
Exists_results: true 0.0005
Exists_results: true 0.0006
```

Проверьте получение данных по роуту «/checkResults» (Рисунок 23).

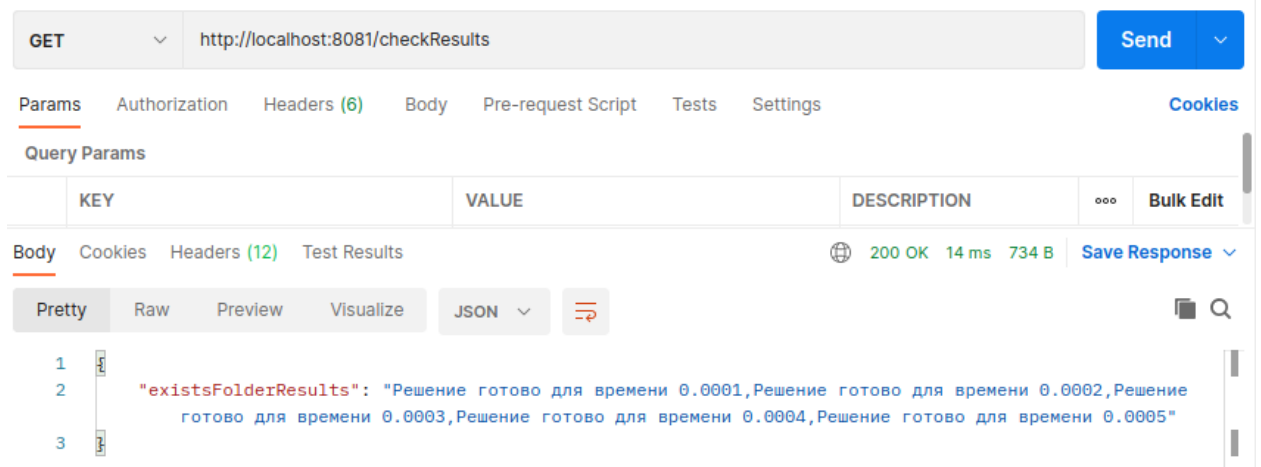


Рисунок 23 – Результат выполнения функции checkResults

На клиенте добавим функцию, которая будет вызываться через определенные промежутки времени и обновлять массив значений с результатами. Время окончания расчета 0.5, поэтому добавим проверку на наличие данного каталога в массиве результатов. При его отсутствии функция через 10 секунд снова начинает проверку.

Листинг 131 – Функция getResults

```
getResults() {  
  axios  
    .get('http://localhost:8081/checkResults/')  
    .then((response) => {  
      let json = response.data;  
      this.existsFolderResults = json.existsFolderResults;  
      if (!json.existsFolderResults.includes("0.5")) {  
        setTimeout(this.getResults, 10000);  
      } else {  
        clearTimeout(this.id_result);  
      }  
      console.log(json);  
    })  
    .catch((error) => console.log(error));  
},
```

После создания процессоров создайте таймаут для проверки результатов.

Листинг 132 – Вызов функции `getResults`

```
if (json.existsProcessors !== "") {  
    clearTimeout(this.id_processor);  
    this.id_result = setTimeout(this.getResults, 10000);  
}
```

Добавьте объявление переменных.

Листинг 133 – Создание переменных

```
existsFolderResults: "",  
id_result: "",
```

В шаблоне создайте новый заголовок и блок с классом `resultFolders`. В цикле в параграфе будем выводить все значения из массива.

Листинг 134 – Вывод результатов на клиента

```
<h2>Выполненный расчет:</h2>  
<div class="resultFolders">  
    <p v-for="res in existsFolderResults.split(',')" :key="res">  
        {{ res }}  
    </p>  
</div>
```

Задайте определенные стили данному блоку: максимальную высоту, стилизация скролла и другие.

Листинг 135 – Создание стилей

```
.resultFolders {  
    max-height: 115px;  
    overflow-y: auto;  
}  
.resultFolders::-webkit-scrollbar {  
    width: 12px;  
    -webkit-box-shadow: inset 0 0 6px rgba(0, 0, 0, 0.3);  
}  
.resultFolders::-webkit-scrollbar-track {  
    -webkit-box-shadow: inset 0 0 6px rgba(0, 0, 0, 0.3);  
    border-radius: 10px;  
}  
.resultFolders::-webkit-scrollbar-thumb {  
    border-radius: 10px;  
    -webkit-box-shadow: inset 0 0 6px rgba(0, 0, 0, 0.5);  
}
```

Проверьте работу приложения в браузере. Выполненный расчет должен отображаться в соответствующем блоке (Рисунок 24).

Подготовка к расчету:

Файлы предыдущего решения успешно удалены.

Файл с входными значениями успешно изменен.

Файлы OpenFOAM успешно изменены.

Расчет успешно запущен. Ожидайте...

Сетка успешно создана

Задача успешно распараллелена

Выполненный расчет:

Решение готово для времени 0.0001

Решение готово для времени 0.0002

Решение готово для времени 0.0003

Решение готово для времени 0.0004

Решение готово для времени 0.0005

Рисунок 24 – Результат расчета

2.6 Создание модального окна

Модальные окна необходимы для вывода сообщений пользователю. Например, по нажатию на кнопку «Остановить расчет» необходимо вывести сообщение пользователю об успешном завершении, либо при обнаружении ошибки во время работы решателя модальное окно сообщит пользователю о невозможности продолжения расчета и так далее.

В каталоге components создайте компонент ModalComponent.vue. Он состоит из заголовка и тела, которые содержат слоты для динамического изменения контента. В заголовке также присутствует кнопка закрытия.

Листинг 136 – Разметка модального окна

```
<template>
  <transition name="modal-fade">
    <div class="modal-backdrop">
      <div class="modal">
        <header class="modal-header">
          <slot name="header" />
          <a class="modal-close btn" @click="close"> x </a>
        </header>
        <section class="modal-body">
          <slot name="body" />
        </section>
      </div>
    </div>
  </transition>
</template>
```

Укажите имя компонента и один метод по закрытию модального окна.

Листинг 137 – Метод закрытия модально окна

```
<script>
export default {
  name: "ModalComponent",
  methods: {
    close() {
      this.$emit("close");
    },
  },
};
</script>
```

Укажите стили для компонента. Его положение будет фиксированным, фон становиться немного темнее.

Листинг 138 – Стили модального окна

```
<style scoped>
.modal-backdrop {
  position: fixed;
  top: 0;
  bottom: 0;
  left: 0;
  right: 0;
  background-color: rgba(0, 0, 0, 0.3);
  display: flex;
  justify-content: center;
  align-items: center;
}

.modal {
  background: #ffffff;
  box-shadow: 2px 2px 20px 1px;
  display: flex;
  flex-direction: column;
  padding: 1rem;
}

.modal-header {
  padding: 1.5rem;
  display: flex;
  position: relative;
  border-bottom: 1px solid #eeeeee;
  color: #4174d3;
  justify-content: space-between;
}

.modal-body {
  position: relative;
  padding: 1.5rem;
}

.btn-close {
  position: absolute;
  top: 0;
  right: 0;
  border: none;
  font-size: 20px;
  cursor: pointer;
  font-weight: bold;
  color: #f52a2a;
  background: transparent;
}

.btn-close:hover {
  box-shadow: none;
}
```

Листинг 139 – Стили модального окна продолжение

```
.modal-fade-enter,  
.modal-fade-leave-to {  
  opacity: 0;  
}  
  
.modal-fade-enter-active,  
.modal-fade-leave-active {  
  transition: opacity 0.5s ease;  
}  
  
.modal-close {  
  color: #aaa;  
  line-height: 50px;  
  font-size: 80%;  
  position: absolute;  
  right: 0;  
  text-align: center;  
  top: 0;  
  width: 70px;  
  text-decoration: none;  
}  
  
.modal-close:hover {  
  color: rgb(211, 89, 89);  
}  
  
.btn {  
  background-color: white;  
  font-weight: bold;  
  padding: 0.5em 0;  
  text-decoration: none;  
  cursor: pointer;  
}  
  
</style>
```

На клиенте в ResultPage.vue импортируйте созданный компонент.

Листинг 140 – Импорт модального окна

```
<script>  
import axios from "axios";  
import ModalComponent from "../ModalComponent.vue";
```

Зарегистрируйте его.

Листинг 141 – Регистрация компонента

```
export default {  
  name: "ResultPage",  
  components: {  
    ModalComponent,  
  },
```

Добавьте в шаблон. Установите атрибут v-show и событие close. В слот header добавьте заголовок, основная информация будет зависеть от переменной stopped.

Листинг 142 – Создание модального окна для остановки расчета

```
<ModalComponent v-show="isModalVisible" @close="closeModal">
  <template v-slot:header> Уведомление </template>
  <template v-slot:body>
    {{
      stopped
      ? "Расчет успешно завершен"
      : "Не удалось остановить расчет"
    }}
  </template>
</ModalComponent>
```

Создайте необходимые переменные.

Листинг 143 – Создание переменных

```
isModalVisible: false,
stopped: false,
```

Создайте метод закрытия модального окна.

Листинг 144 – Метод closeModal

```
methods: {
  closeModal() {
    this.isModalVisible = false;
  },
}
```

Добавьте обработчик события click для кнопки остановки расчета.

Листинг 145 – Создание события click

```
<button class="button" @click="stopCalc()">Остановить расчет</button>
```

Создайте соответствующую функцию. При остановке расчета модальное окно изменяет видимость, останавливаются все таймауты, переменной stopped присваивается значение true. Отправляется POST-запрос по URL «/stopCalc».

Листинг 146 – Функция stopCalc

```
stopCalc() {  
  this.isModalVisible = true;  
  clearTimeout(this.id);  
  clearTimeout(this.id_processor);  
  clearTimeout(this.id_result);  
  console.log("stopped");  
  this.stopped = true;  
  axios  
    .post("http://localhost:8081/stopCalc/", { stop: "stop" })  
    .then((response) => {  
      console.log(response);  
    })  
    .catch((error) => {  
      console.log(error);  
    });  
},
```

На сервере в файле Index.js создайте переменную stopped.

Листинг 147 – Создание переменной

```
let stopped = "";
```

Объявите обработку GET-запроса.

Листинг 148 – GET-запрос по URL «/stopCalc»

```
app.get("/stopCalc", function (req, res, next) {  
  res.send({ stop: `${stopped}` });  
  next();  
});
```

При получении в качестве ответа значение req.body.stop, равным строке stop, выполнение передается функции exes, которая переходит в домашнюю директорию и вызывает уничтожение всех процессов решателя rhoPimpleFoam. Результат выполнения отображается в терминале. При успешном выполнении выводится сообщение killed, иначе сообщение not killed.

Листинг 149 – POST-запрос по URL «/stopCalc»

```
app.post("/stopCalc", urlencodedParser, function (req, res) {
  if (!req.body) return res.sendStatus(400);
  console.log(req.body);
  stopped = req.body.stop;
  if (req.body.stop === "stop") {
    const script1 = exec(
      "cd /home/sveta/; killall rhoPimpleFoam",
      (error, stdout, stderr) => {
        console.log(stdout);
        console.log(stderr);
        if (error !== null) {
          console.log(`exec error: ${error}`);
        }
      }
    );
    if (script1 == -1) {
      console.log("not killed");
    } else {
      console.log("killed");
    }
  }
});
```

В функцию clearAllPathAndVars добавьте сброс переменных.

Листинг 150 – Сброс значений переменных

```
id_results = "";
stopped = "";
console.log("clearAllPathAndVars");
}
```

В функции runSh при завершении выполнения добавьте удаление всех таймаутов.

Листинг 151 – Удаление таймаутов при завершении выполнения

```
spawnRun.on("close", (code) => {
  clearTimeout(id);
  clearTimeout(id_processors);
  clearTimeout(id_results);
  console.log(`child process runSh exited with code ${code}`);
});
```

То же самое выполните для события при обнаружении ошибки.

Листинг 152 – Удаление таймаутов при обнаружении ошибки

```
spawnRun.on("error", (error) => {  
    clearTimeout(id);  
    clearTimeout(id_processors);  
    clearTimeout(id_results);  
    runned = "Ошибка при выполнении расчета";  
    console.log(`error: ${error.message}`);  
});
```

Добавьте условие для завершения выполнения расчета в функцию проверки построения сетки. При ошибке построения переменной stopped присвойте значение stop.

Листинг 153 – Изменение функции проверки построения сетки

```
if (existsMesh) {  
    meshCreated = "Сетка успешно создана";  
    clearTimeout(id);  
    id_processors = setTimeout(checkProcessors, 4000);  
} else if (stopped === "stop") {  
    clearTimeout(id);  
} else if (meshCreatedAbort) {  
    meshCreated =  
    | "Сетка не может быть построена с данными входными параметрами.";  
    console.log(meshCreated);  
    clearTimeout(id);  
    stopped = "stop";  
} else setTimeout(createMesh, 4000);
```

Аналогично добавьте условие остановки выполнения функции проверки распараллеливания задачи.

Листинг 154 – Изменение функции проверки распараллеливания задачи

```
if (existsProcessors) {  
    processorsCreated = "Задача успешно распараллелена";  
    clearTimeout(id_processors);  
    id_results = setTimeout(checkResults, 10000);  
} else if (stopped === "stop") {  
    clearTimeout(id_processors);  
} else  
    setTimeout(checkProcessors, 4000);
```

Также добавьте условие в функцию проверки результатов расчета.

Листинг 155 – Изменение функции проверки результатов расчета

```
if (existsFolderResults) {  
  existsResults.push(`Решение готово для времени ${folderTime}`);  
  clearTimeout(id_results);  
  folderTime += 0.0001;  
  folderTime = +folderTime.toFixed(4);  
  if (folderTime < 0.5) {  
    id_results = setTimeout(checkResults, 10000);  
  }  
} else if (stopped === "stop") {  
  clearTimeout(id_results);  
} else setTimeout(checkResults, 10000);
```

Запустите расчет и остановите его до создания сетки (Рисунок 25).

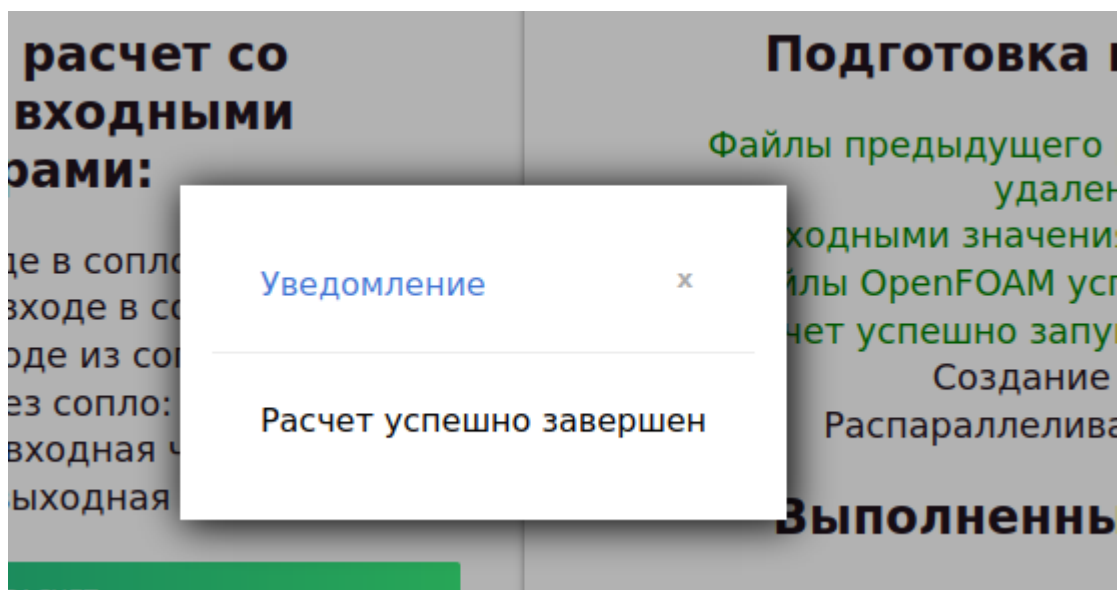


Рисунок 25 – Завершение расчета

Вывод в терминал должен быть следующим. Выполнение расчета еще не было начато, поэтому процесс rhoPimpleFoam еще не запустился.

Листинг 156 – Вывод в терминал сообщения об остановке расчета

```
{ stop: 'stop' }  
killed  
  
rhoPimpleFoam: процесс не найден  
  
exec error: Error: Command failed: cd /home/sveta  
/; killall rhoPimpleFoam  
rhoPimpleFoam: процесс не найден  
  
Mesh exists: false
```

Теперь заново запустите расчет, дождитесь выполнения хотя бы для одного времени (Рисунок 26).

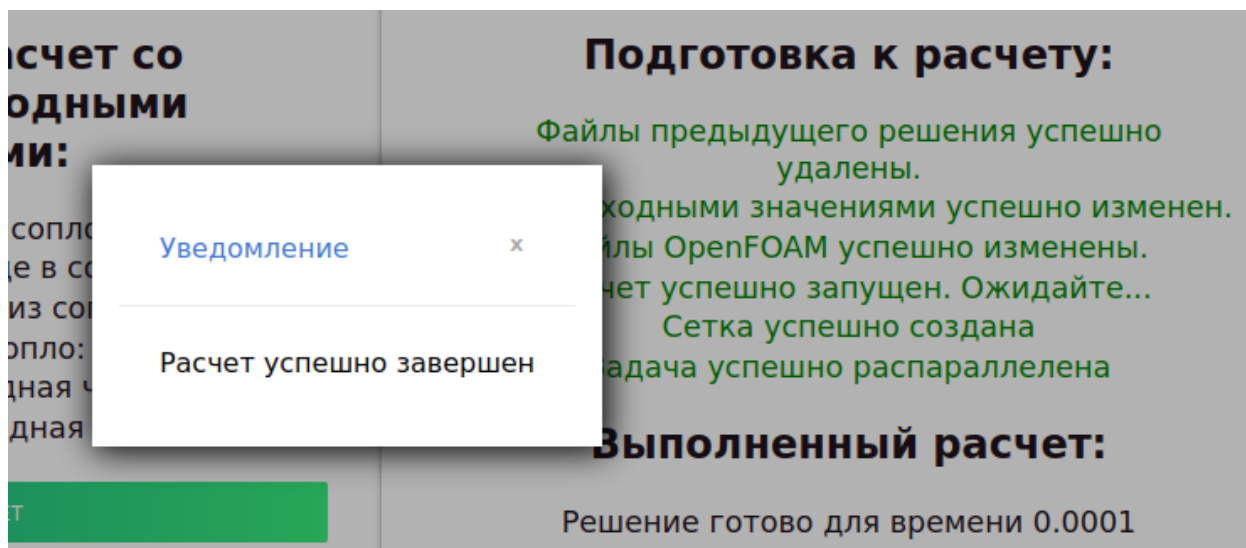


Рисунок 26 – Завершение расчета

Вывод в терминал должен быть следующим.

Листинг 157 – Вывод в терминал сообщения об остановке расчета

```
Exists_processors: true
Exists_results: true 0.0001
{ stop: 'stop' }
killed
```

```
stderr run: -----
-----
Primary job terminated normally, but 1 process returned
a non-zero exit code. Per user-direction, the job has been aborted.
-----
```

```
stderr run: -----
-----
mpirun noticed that process rank 0 with PID 0 on node sveta-VirtualBox exited
on signal 15 (Terminated).
-----
```

```
Exists_results: false 0.0002
```

Остановку процессов можно выполнять, используя объявленный выше controller.

Теперь введите следующие входные параметры (Рисунок 27).

Давление воздуха P_0 на входе в сопло: **2000000** (Па)
 Температура воздуха T_0 на входе в сопло: **1000** (K)
 Давление воздуха P_1 на выходе из сопла: **8000** (Па)
 Расход воздуха G через сопло: **4** (Kg/c)
 Угол раскрытия сопла α - входная часть: **14** (°)
 Угол раскрытия сопла β - выходная часть: **22** (°)

Рисунок 27 – Входные параметры

Сетка с такими параметрами будет построена, однако решатель завершит свое выполнение с ошибкой в самом начале. Вывод в терминал будет следующим.

Листинг 158 – Ошибка выполнения расчета

```
stderr run: [0]
[0]
[0] --> FOAM FATAL ERROR:
[0] Negative initial temperature T0: -1776.500457
```

Добавьте обработчик ошибки в функцию runSh. При ее возникновении переменной runAbort будет присвоено значение true, расчет будет завершен.

Листинг 159 – Добавление обработчика ошибки решателя

```
spawnRun.stderr.on("data", (data) => {
  console.log(`stderr run: ${data}`);
  if (
    data.includes(
      'ERROR:salome'
    )
  ) {
    meshCreatedAbort = true;
    createMesh();
  }
  if (data.includes("Foam::error")) {
    runAbort = true;
    console.log(`Расчет завешен с ошибкой`);
    stopped = "stop";
  }
});
```

Создайте соответствующий роут, отвечающий за возникновение ошибки на этапе расчета.

Листинг 160 – Использование роута «/runAbort»

```
app.use("/runAbort", function (req, res, next) {
  res.send({ runAbort: `${runAbort}` });
  console.log(`runAbort: ${runAbort}`);
  next();
});
```

Объявите переменную.

Листинг 161 – Создание переменной

```
let runAbort = false;
```

Добавьте сброс значения в функцию `clearAllPathAndVars`.

Листинг 162 – Сброс значений к дефолтным

```
runAbort = false;  
console.log("clearAllPathAndVars");
```

Проверьте корректность получения данных при возникновении ошибки (Рисунок 28).

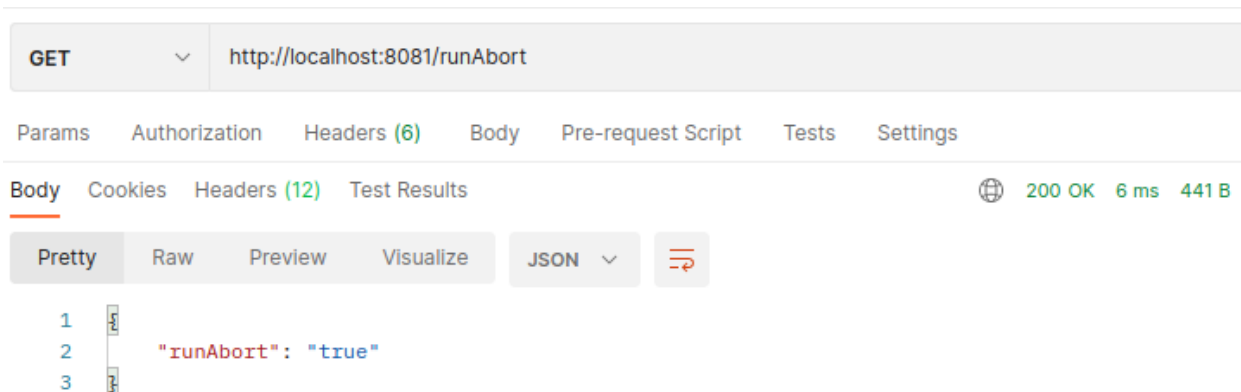


Рисунок 28 – Результат выполнения функции

Создайте новое модальное окно, задайте атрибут `v-show` и событие `close`. В слот `body` введите сообщение.

Листинг 163 – Создание модального окна об ошибке при выполнении расчета

```
<ModalComponent v-show="isModalVisible2" @close="closeModal2">  
  <template v-slot:header> Уведомление </template>  
  <template v-slot:body>  
    Расчет не может быть продолжен. Введите другие параметры.  
  </template>  
</ModalComponent>
```

Добавьте переменную.

Листинг 164 – Добавление переменной

```
isModalVisible2: false,
```

Создайте функцию закрытия модального окна.

Листинг 165 – Функция закрытия модального окна

```
closeModal2() {  
  this.isModalVisible2 = false;  
},
```

Можете не выносить в отдельную функцию, а сразу записать в обработчике.

Добавьте метод `getErrors`, выполняющий GET-запрос и получающий значение наличия ошибки при выполнении расчета. Также добавьте остановку выполнения функции при завершении расчета.

Листинг 166 – Функция `getErrors`

```
getErrors() {
  if (this.stopped) {
    clearTimeout(this.id_errors);
    return;
  }
  axios
    .get(`http://localhost:8081/runAbort/`)
    .then((response) => {
      let json = response.data;
      if (json.runAbort == "true") {
        this.isModalVisible2 = true;
        this.stopped = true;
        clearTimeout(this.id_errors);
      } else {
        setTimeout(this.getErrors, 10000);
        console.log(json);
      }
    })
    .catch((error) => console.log(error));
}
```

В функцию `createProcessors` после запуска таймаута получения результатов запустите еще один таймаут для получения ошибки.

Листинг 167 – Добавление таймаута

```
createdProcessors() {
  axios
    .get(`http://localhost:8081/checkProcessors/`)
    .then((response) => {
      let json = response.data;
      this.checkProcessors = json.existsProcessors;
      if (json.existsProcessors !== "") {
        clearTimeout(this.id_processor);
        this.id_result = setTimeout(this.getResults, 10000);
        this.id_errors = setTimeout(this.getErrors, 10000);
      } else {
        setTimeout(this.createdProcessors, 4000);
      }
      console.log(json);
    })
    .catch((error) => console.log(error));
},
```

Объявите переменную.

Листинг 168 – Создание переменной

```
id_errors: "",
```

Выполните расчет с некорректными данными. Должно появиться модальное окно, расчет должен остановиться (Рисунок 29).

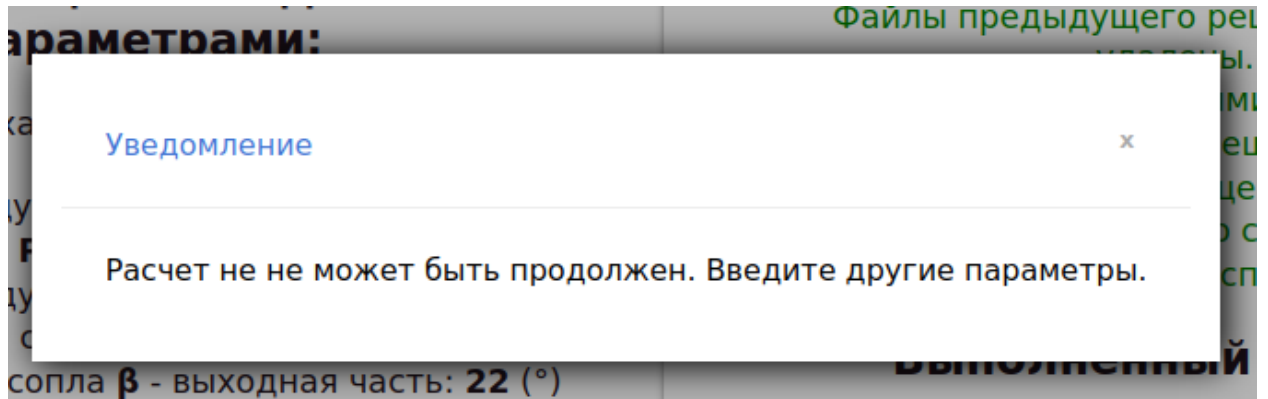


Рисунок 29 – Ошибка выполнения расчета

Создайте еще два модальных окна, которые будут отвечать за открытие файла log. Создадим определенное условие: если задача не была еще распараллелена, то выведем одно модальное окно с сообщением «Дождитесь распараллеливания расчета», иначе выведем другое модальное окно со значением переменной `openLogFile`. Заполните соответствующие слоты для каждого окна, задайте условие видимости и событие закрытия окна. Последнее можно прописать сразу в директиве `@close`, не создавая дополнительный метод.

Листинг 169 – Создание модальных окон

```
<ModalComponent v-show="isModalVisible3" @close="this.isModalVisible3 = false">
  <template v-slot:header> Уведомление </template>
  <template v-slot:body>
    {{ openedLogFile || "Файл log открывается. Ожидайте..." }}
  </template>
</ModalComponent>

<ModalComponent v-show="isModalVisible4" @close="this.isModalVisible4 = false">
  <template v-slot:header> Уведомление </template>
  <template v-slot:body>
    Дождитесь распараллеливания расчета.
  </template>
</ModalComponent>
```

Задайте модели для использованных переменных.

Листинг 170 – Добавление переменных

```
isModalVisible3: false,  
isModalVisible4: false,  
openedLogFile: "",
```

Добавьте выполнение функции при нажатии на кнопку открытия файла log.

Листинг 171 – Добавление события click

```
<button class="button" @click="openLogFile()">  
| Открыть файл log для просмотра решения  
</button>
```

Создайте соответствующую функцию. Изначально идет проверка на создание каталогов processor*. Если они еще не созданы, то показывается модальное окно 4, иначе модальное окно 3. Выполняется POST-запрос по URL «/logFile». При успешном выполнении запроса вызывается следующая функция logFileOpened, которая выполняет GET-запрос по URL «/logFileOpened» и присваивает ответ от сервера переменной openedLogFile.

Листинг 172 – Функция openLogFile

```
openLogFile() {  
  if (this.checkProcessors === "") {  
    this.isModalVisible4 = true;  
  } else {  
    this.isModalVisible3 = true;  
    axios  
      .post("http://localhost:8081/logFile/", { logFile: "open" })  
      .then((response) => {  
        console.log(response);  
        this.logFileOpened();  
      })  
      .catch((error) => {  
        console.log(error);  
      });  
  }  
},
```

Листинг 173 – Функция logFileOpened

```
logFileOpened() {  
  axios  
    .get(`http://localhost:8081/logFileOpened/`)  
    .then((response) => {  
      let json = response.data;  
      this.openedLogFile = json.logFileOpened;  
      console.log(json);  
    })  
    .catch((error) => console.log(error));  
},
```

На сервере создайте обработчики запросов для URL «/logFile» и «/logFileOpened». При получении из тела запроса значение logFile, равным «open», вызывается функция openLogFile.

Листинг 173 – Использование роутов «/logFile» и «/logFileOpened»

```
app.post("/logFile", function (req, res) {  
  logFile = req.body.logFile;  
  if (logFile === "open") {  
    console.log("Open LogFile");  
    openLogFile();  
  } else {  
    console.log("No open LogFile");  
  }  
  res.send(req.body);  
});  
  
app.get("/logFile", jsonParser, function (req, res) {  
  res.send({ logFile: `${logFile}` });  
});  
  
app.use("/logFileOpened", function (req, res, next) {  
  res.send({ logFileOpened: `${logFileOpened}` });  
  next();  
});
```

Объявите переменные, которые будут использоваться.

Листинг 174 – Объявление переменных

```
let logFile = "";  
let logFileOpened = "";  
let existsLog = "";
```

Добавьте их в функцию clearAllPathAndVars

Листинг 175 – Сброс значений к дефолтным

```
    logFile = "";
    logFileOpened = "";
    existsLog = "";
    console.log("clearAllPathAndVars");
}
```

Создайте функцию openLogFile. Для начала необходимо выполнить проверку на существование файла log. Задайте вывод в консоль результат выполнения. Если файл существует, то с использованием дочерней функции exec переходим в каталог решения OpenFOAM и вызываем текстовый редактор gedit. В зависимости от результата выполнения функции exec переменной logFileOpened присваивается значение. Если файла не существует, то функция exec не вызывается, а переменной logFileOpened присваивается значение «Файл не существует».

Листинг 176 – Функция openLogFile

```
async function openLogFile() {
  try {
    existsLog = fs.existsSync(`/home/sveta/OpenFOAM/sveta-9/run/nozzle_1/log`);
    console.log("Exists_log: ", existsLog);
  } catch (e) {
    console.log(e);
  }
  if (existsLog) {
    exec(
      "cd /home/sveta/OpenFOAM/sveta-9/run/nozzle_1/; gedit log",
      (error, stdout, stderr) => {
        if (error) {
          console.error(`exec error: ${error}`);
          logFileOpened = "Не удалось открыть файл";
          return;
        }
        console.log(`stdout: ${stdout}`);
        console.error(`stderr: ${stderr}`);
        logFileOpened = "Файл log открывается. Ожидайте...";
      }
    );
  } else {
    logFileOpened = "Файл не существует";
  }
}
```

Теперь создайте еще два модальных окна, которые будут отвечать за

открытие paraView. Задайте им директивы v-show и событие закрытия окна. В первое модальное окно будет выводиться информация об открытии paraView, во второе предупреждение, что расчет должен быть выполнен хотя бы для одного значения времени.

Листинг 177 – Создание модальных окон

```
<ModalComponent v-show="isModalVisible5" @close="this.isModalVisible5 = false">
  <template v-slot:header> Уведомление </template>
  <template v-slot:body>
    {{ paraViewStarted }}
  </template>
</ModalComponent>
<ModalComponent v-show="isModalVisible6" @close="this.isModalVisible6 = false">
  <template v-slot:header> Предупреждение </template>
  <template v-slot:body>
    Дождитесь выполнения расчета хотя бы для одного значения времени.
    Расчет отображается на правой панели.
  </template>
</ModalComponent>
```

Добавьте обработчик события click соответствующей кнопке.

Листинг 178 – Добавление директивы v-click

```
<button class="button" @click="openParaView()">
  Открыть ParaView для просмотра текущих результатов
</button>
```

Создайте функцию, выполняющую проверку на наличие результатов и осуществляющую POST-запрос для открытия paraView.

Листинг 179 – Функция openParaView

```
openParaView() {
  if (this.existsFolderResults === "") {
    this.isModalVisible6 = true;
  } else {
    this.isModalVisible5 = true;
    axios
      .post("http://localhost:8081/paraView/", { paraView: "run" })
      .then((response) => {
        console.log(response);
        this.startParaView();
      })
      .catch((error) => {
        console.log(error);
      });
  }
},
```

Создайте функцию `startParaView`, которая будет вызываться после успешного выполнения POST-запроса и получать ответ об открытии `paraView`.

Листинг 180 – Функция `startParaView`

```
startParaView() {  
  axios  
    .get(`http://localhost:8081/paraViewStarted/`)  
    .then((response) => {  
      let json = response.data;  
      this.paraViewStarted = json.paraViewStarted;  
      console.log(json);  
    })  
    .catch((error) => console.log(error));  
},
```

Объявите необходимые переменные.

Листинг 181 – Объявление переменных

```
isModalVisible5: false,  
isModalVisible6: false,  
paraViewStarted: "",
```

По аналогии создайте обработчики запросов для открытия `paraView`.

Листинг 182 – Создание роутов «`/paraView`» и «`/paraViewStarted`»

```
app.post("/paraView", function (req, res) {  
  console.log(req.body);  
  runParaView = req.body.paraView;  
  if (runParaView === "run") {  
    console.log("Open ParView");  
    startParaView();  
  } else {  
    console.log("not run");  
  }  
  res.send(req.body);  
});  
  
app.get("/paraView", jsonParser, function (req, res) {  
  res.send({ paraView: `${runParaView}` });  
});  
  
app.use("/paraViewStarted", function (req, res, next) {  
  res.send({ paraViewStarted: `${paraViewStarted}` });  
  next();  
});
```

Создайте функцию `startParaView`, которая будет запускать функцию `spawn`. Подпишитесь на события. После перехода в директорию проекта выполняется команда `reconstructPar` для объединения результатов расчета из всех каталогов `processor*`. Затем вызывается команда `paraFoam` для создания временного файла с расширением `.OpenFOAM` для `paraView`.

Листинг 183 – Функция `startParaView`

```
async function startParaView() {
  const ls = spawn(
    "cd /home/sveta/OpenFOAM/sveta-9/run/nozzle_1/reconstructPar; paraFoam;",
    [],
    { shell: true }
  );

  ls.stdout.on("data", (data) => {
    console.log(`stdout: ${data}`);
  });

  ls.stderr.on("data", (data) => {
    console.log(`stderr: ${data}`);
  });

  ls.on("error", (error) => {
    paraViewStarted = "Не удалось запустить ParaView.";
    console.log(`error: ${error.message}`);
  });

  ls.on("spawn", () => {
    paraViewStarted = "ParaView успешно запущен. Ожидайте...";
    console.log(`ParaView runned`);
  });

  ls.on("close", (code) => {
    console.log(`child process exited with code ${code}`);
  });
}
```

Если не вызывать утилиту `reconstructPar`, то в настройках `paraView` следует выбирать `decomposed Case` для поля `Case Type`, предварительно необходимо создать файл с расширением `.foam` (например, команда в терминале: `touch foam.foam`), затем открыть `paraView` (команда: `paraview`), в нем в стандартном меню выбрать данный файл.

Объявите использованные переменные.

Листинг 184 – Объявление переменных

```
let runParaView = "";  
let paraViewStarted = "";
```

Добавьте переменные в функцию clearAllPathAndVars.

Листинг 185 – Сброс значений к дефолтным

```
runParaView = "";  
paraViewStarted = "";  
console.log("clearAllPathAndVars");  
}
```

Измените файл App.vue. По нажатию на кнопку «Новый расчет» необходимо выполнять переход на главную страницу и вызывать функцию newCalc, которая выполнит POST-запрос по URL «/stopCalc».

Листинг 186 – Добавление обработчика события click

```
<button class="btn" @click="$router.push('/'), newCalc()">  
  Новый расчет  
</button>
```

В script добавьте импорт axios и создайте метод newCalc.

Листинг 187 – Создание функции newCalc

```
<script>  
import axios from "axios";  
  
export default {  
  name: 'App',  
  methods: {  
    newCalc() {  
      console.log("new calc");  
      axios  
        .post("http://localhost:8081/stopCalc/", { stop: "stop" })  
        .then((response) => {  
          console.log(response);  
        })  
        .catch((error) => {  
          console.log(error);  
        });  
    },  
  },  
}
```

В функцию stopCalc файла index.js добавьте остановку всех таймеров в условие if (req.body.stop === "stop").

Листинг 188 – Остановка таймеров в index.js

```
clearTimeout(id)
clearTimeout(id_processors)
clearTimeout(id_results)
```

В компонент ResultPage.vue необходимо добавить функцию, которая будет отслеживать роут «/stopCalc», и если значение будет равным stop, то вызовет функцию stopCalc для остановки всех таймеров и завершения расчета.

Листинг 189 – Функция getStop

```
getStop() {
  axios
    .get(`http://localhost:8081/stopCalc/`)
    .then((response) => {
      let json = response.data;
      if (json.stop === "stop") {
        this.stopCalc()
        clearTimeout(this.id_stop)
      }
      else
      {
        this.id_stop = setTimeout(this.getStop, 5000);
      }
    })
    .catch((error) => console.log(error));
},
```

В хук жизненного цикла добавьте таймер для запуска данной функции.

Листинг 190 – Объявление таймера

```
created() {
  this.getParams();
  this.id_stop = setTimeout(this.getStop, 5000);
},
```

Объявите переменную.

Листинг 191 – Объявление переменной

```
id_stop: "",
```

Добавьте в функцию getResultResults условие остановки таймера.

Листинг 192 – Условие остановки таймера

```
getResults() {  
    if (this.stopped) {  
        clearTimeout(this.id_result);  
        return;  
    }  
}
```

Добавьте аналогичное условие в функцию createdProcessors.

Листинг 193 – Условие остановки таймера

```
createdProcessors() {  
    if (this.stopped) {  
        clearTimeout(this.id_processor);  
        return;  
    }  
}
```

Добавьте последнее условие остановки таймера на этапе проверки создания сетки.

Листинг 194 – Условие остановки таймера

```
createdMesh() {  
    if (this.stopped) {  
        clearTimeout(this.id);  
        return;  
    }  
}
```

Проверьте работу приложения. Убедитесь, что все таймеры останавливаются при запуске нового расчета. Никаких логов в терминале не должно выводиться ни в VS Code, ни в браузере.

На этом создание веб-приложения завершено.

ЗАКЛЮЧЕНИЕ

В ходе данной работы было создано веб-приложение по расчету сопла Лаваля в OpenFOAM, изменены скрипты, необходимые перед выполнением расчета, рассмотрены варианты запуска внешних функций.

Вопросы для самоконтроля:

1. Чем отличается функция `spawn` от функции `exec`? В каких случаях используется первая, а в каких вторая?
2. С помощью какой команды скрипту добавляется право на выполнение?
3. Как определить область начального распределения? Как оно задается и в каком файле?
4. Как импортируются компоненты во Vue.js?
5. Как осуществляется обмен данными между клиентом и сервером?